

YOLOv8 Pipeline (Colab, Function-Based)

1. Business Problem
2. Dataset Description
3. Kaggle Dataset Download
4. Environment Setup
5. Dataset Validation
6. EDA & Visualization
7. Coordinate Transformation
8. Bounding Box Visualization
9. YOLOv8 Training
10. Model Evaluation
11. Inference Pipeline
12. License Plate Blurring
13. Before vs After Comparison
14. Error Analysis
15. Recommendations
16. Conclusion

Dataset Info dataset/

| — images/

| | — train/

| | — val/

| | — test/

| — labels/

| | — train/

| | — val/

| | — test/

SECTION 1 — Business Problem

Business Problem

With the rapid growth of CCTV systems, smart traffic monitoring, and surveillance applications, organizations continuously collect vehicle images and videos.

These images often contain Personally Identifiable Information (PII), particularly license plate numbers, creating privacy and compliance risks under regulations such as GDPR and CCPA.

Manual anonymization of license plates is:

- time-consuming,

- expensive,
- non-scalable.

The objective of this project is to build an automated real-time license plate detection and blurring system using YOLOv8.

The system should:

- accurately detect license plates,
- blur only the sensitive regions,
- preserve the usability of the remaining visual data,
- support real-time deployment in surveillance systems.

SECTION 2 — Dataset Description

Dataset Description

The dataset contains annotated vehicle images for license plate detection.

Dataset Splits

Split	Images
Train	25,470
Validation	1,000
Test	400

Annotation Format

Annotations are stored in YOLO format:

Coordinates are normalized between 0 and 1.

SECTION 3 — Environment Setup

Install Dependencies

```
def install_dependencies():  
    !pip install ultralytics -q  
install_dependencies()  
----- 1.2/1.2 MB 19.1 MB/s eta  
0:00:00a 0:00:01
```

Import Libraries

```
def import_libraries():

    global os
    global cv2
    global yaml
    global torch
    global random
    global np
    global pd
    global plt
    global YOLO

    import os
    import cv2
    import yaml
    import torch
    import random
    import numpy as np
    import pandas as pd
    import matplotlib.pyplot as plt

    from ultralytics import YOLO

import_libraries()

Creating new Ultralytics Settings v0.0.6 file []
View Ultralytics Settings with 'yolo settings' or at
'/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings
runs_dir=path/to/dir'. For help see
https://docs.ultralytics.com/quickstart/#ultralytics-settings.

!sudo apt-get install tesseract-ocr -y
!pip install pytesseract -q

Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
tesseract-ocr is already the newest version (4.1.1-2.1build1).
0 upgraded, 0 newly installed, 0 to remove and 133 not upgraded.
```

GPU Verification

```
def check_gpu():

    print("GPU Available:", torch.cuda.is_available())

    if torch.cuda.is_available():
        print("GPU Name:", torch.cuda.get_device_name(0))
```

```
check_gpu()
```

```
GPU Available: True
```

```
GPU Name: Tesla T4
```

Reproducibility

```
def set_seed(seed=42):
```

```
    random.seed(seed)
```

```
    np.random.seed(seed)
```

```
    torch.manual_seed(seed)
```

```
set_seed()
```

SECTION 4 — Kaggle Dataset Loading

```
def get_dataset_paths(base_path):
```

```
    paths = {
```

```
        "base_path": base_path,
```

```
        "train_images": f"{base_path}/images/train",
```

```
        "val_images": f"{base_path}/images/val",
```

```
        "test_images": f"{base_path}/images/test",
```

```
        "train_labels": f"{base_path}/labels/train",
```

```
        "val_labels": f"{base_path}/labels/val",
```

```
        "test_labels": f"{base_path}/labels/test",
```

```
    }
```

```
    return paths
```

```
base_path = "/kaggle/input/datasets/gangadharpanda/license-plate-  
detection-and-blurring/License Plate Detection and Blurring"
```

```
paths = get_dataset_paths(base_path)
```

```
print(paths)
```

```
{'base_path': '/kaggle/input/datasets/gangadharpanda/license-plate-  
detection-and-blurring/License Plate Detection and Blurring',  
'train_images': '/kaggle/input/datasets/gangadharpanda/license-plate-  
detection-and-blurring/License Plate Detection and  
Blurring/images/train', 'val_images':  
'/kaggle/input/datasets/gangadharpanda/license-plate-detection-and-  
blurring/License Plate Detection and Blurring/images/val',  
'test_images': '/kaggle/input/datasets/gangadharpanda/license-plate-  
detection-and-blurring/License Plate Detection and
```

```
Blurring/images/test', 'train_labels':  
'/kaggle/input/datasets/gangadharpanda/license-plate-detection-and-  
blurring/License Plate Detection and Blurring/labels/train',  
'val_labels': '/kaggle/input/datasets/gangadharpanda/license-plate-  
detection-and-blurring/License Plate Detection and  
Blurring/labels/val', 'test_labels':  
'/kaggle/input/datasets/gangadharpanda/license-plate-detection-and-  
blurring/License Plate Detection and Blurring/labels/test'}
```

SECTION 5 — Create YOLO YAML File

```
def create_yaml_file(base_path):  
    yaml_content = {  
        "path": base_path,  
        "train": "images/train",  
        "val": "images/val",  
        "test": "images/test",  
        "names": {  
            0: "license_plate"  
        }  
    }  
    yaml_path = "/kaggle/working/data.yaml"  
    with open(yaml_path, "w") as f:  
        yaml.dump(yaml_content, f)  
    print("YAML File Created")  
    return yaml_path  
  
yaml_path = create_yaml_file(paths["base_path"])  
print("yaml path:", yaml_path)  
  
YAML File Created  
yaml path: /kaggle/working/data.yaml
```

SECTION 6 — Dataset Validation

Validate Dataset Structure

```
def validate_dataset(paths):  
    print("="*50)
```

```

    for key in paths:
        if key != "base_path":
            print(f"{key}: {len(os.listdir(paths[key]))}")

    print("="*50)

validate_dataset(paths)

=====
train_images: 25625
val_images: 1073
test_images: 386
train_labels: 25672
val_labels: 1073
test_labels: 386
=====

```

Verify Image-Label Mapping

```

def verify_image_label_pairs(image_dir, label_dir):
    image_files = sorted([
        os.path.splitext(f)[0]
        for f in os.listdir(image_dir)
    ])

    label_files = sorted([
        os.path.splitext(f)[0]
        for f in os.listdir(label_dir)
    ])

    missing_labels = set(image_files) - set(label_files)

    print("Missing Labels:", len(missing_labels))

verify_image_label_pairs(
    paths["train_images"],
    paths["train_labels"]
)

Missing Labels: 155

```

SECTION 7 — EDA & Visualization

Show Sample Images

```

def show_sample_images(image_dir, n=6):

```

```
images = random.sample(
    os.listdir(image_dir),
    n
)

plt.figure(figsize=(15,10))

for i, img_name in enumerate(images):
    img_path = os.path.join(
        image_dir,
        img_name
    )

    img = cv2.imread(img_path)

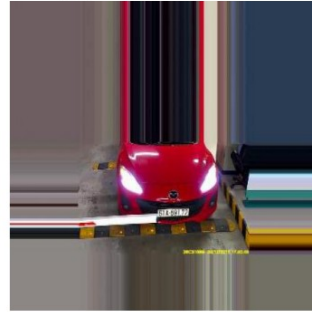
    img = cv2.cvtColor(
        img,
        cv2.COLOR_BGR2RGB
    )

    plt.subplot(2,3,i+1)
    plt.imshow(img)
    plt.title(img_name)
    plt.axis("off")

plt.show()

show_sample_images(paths["train_images"])
```

CarLongPlateGen160_jpg.rf.dd3e1172077f5c5cae9cad27c8f89d5e.jpg



11435227235c29e5.jpg



030aeb1dac498d8f.jpg



xemay637_jpg.rf.a2f35e509d8622fdb3ce49cca350dda0.jpg



1fb6b60b547433a3.jpg



xemay84_jpg.rf.7f1384dd839c12ac9fa03e8a644379e8.jpg



Image Dimension Distribution

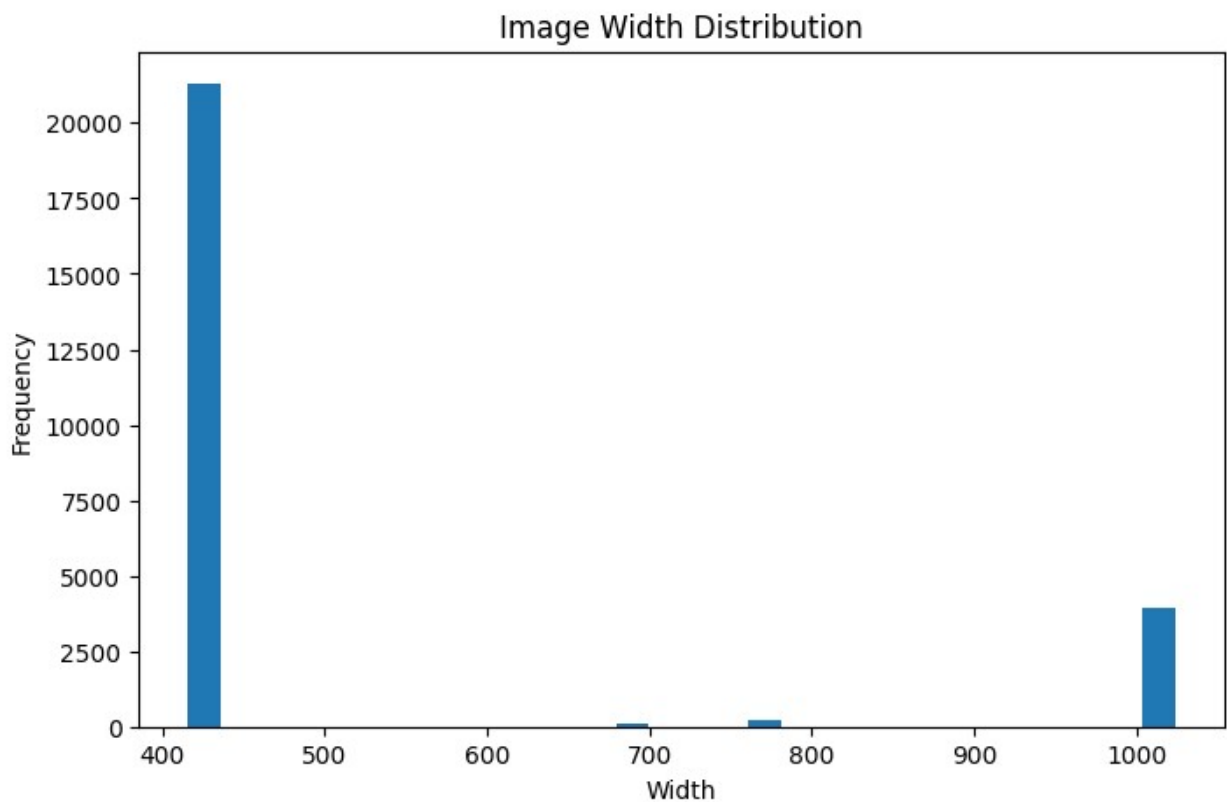
Purpose

Analyzes:

- varying resolutions,
- aspect ratio consistency,
- resizing requirements.

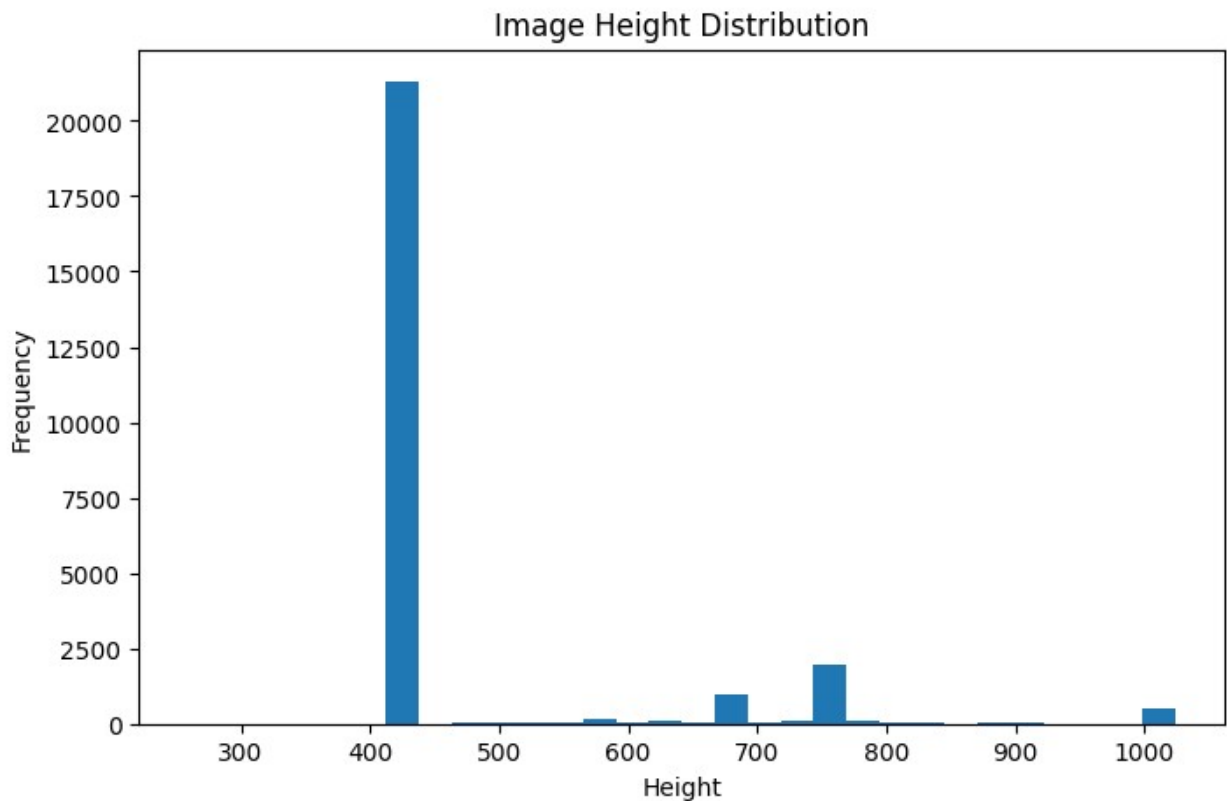
```
def analyze_image_dimensions(image_dir):  
    widths = []  
    heights = []  
  
    for image_name in os.listdir(image_dir):  
        image_path = os.path.join(  
            image_dir,  
            image_name  
        )  
  
        img = cv2.imread(image_path)  
  
        h, w = img.shape[:2]  
  
        widths.append(w)  
        heights.append(h)
```

```
    return widths, heights
img_widths, img_heights = analyze_image_dimensions(
    paths["train_images"]
)
plt.figure(figsize=(8,5))
plt.hist(img_widths, bins=30)
plt.title("Image Width Distribution")
plt.xlabel("Width")
plt.ylabel("Frequency")
plt.show()
```



```
plt.figure(figsize=(8,5))
plt.hist(img_heights, bins=30)
plt.title("Image Height Distribution")
```

```
plt.xlabel("Height")
plt.ylabel("Frequency")
plt.show()
```



Analyze Bounding Boxes

```
def analyze_bounding_boxes(label_dir):
    widths = []
    heights = []

    for file in os.listdir(label_dir):
        file_path = os.path.join(
            label_dir,
            file
        )

        with open(file_path, "r") as f:
            for line in f.readlines():
                _, _, _, w, h = map(
```

```

        float,
        line.strip().split()
    )

    widths.append(w)
    heights.append(h)

    return widths, heights

widths, heights = analyze_bounding_boxes(
    paths["train_labels"]
)

```

Plot Bounding Box Distribution

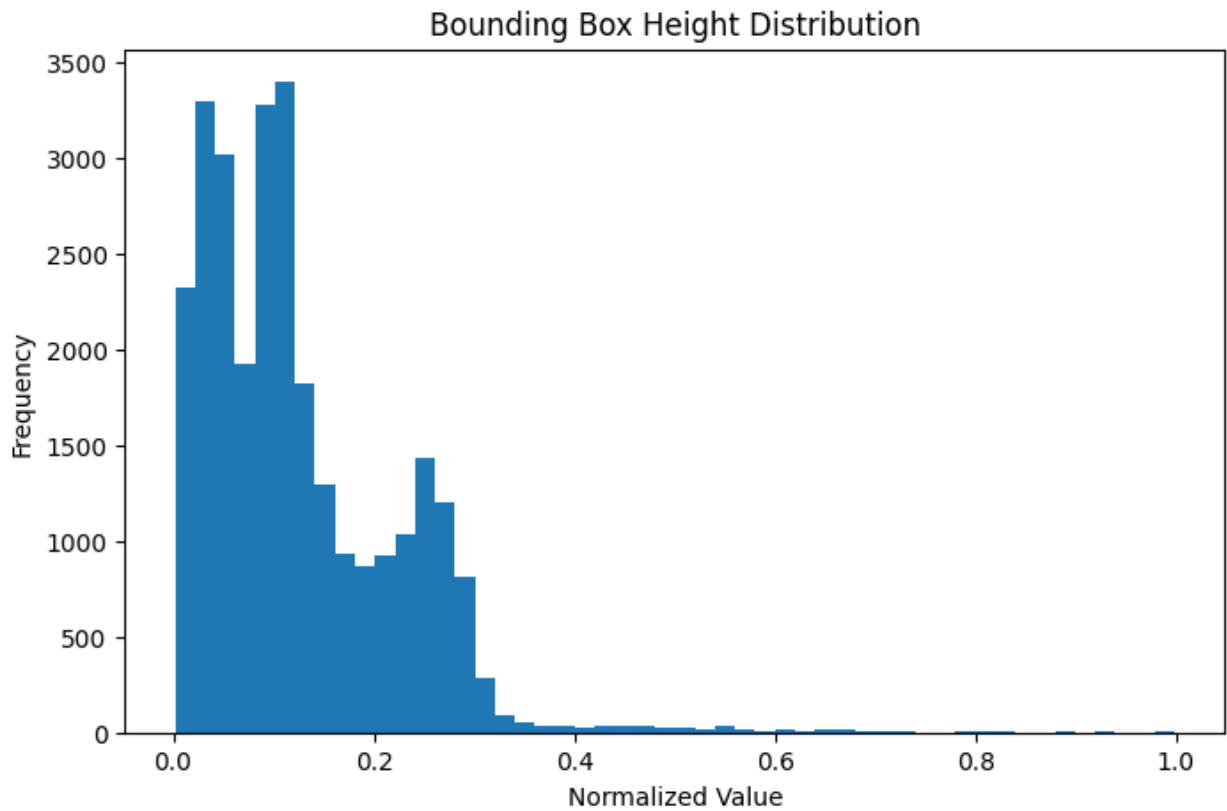
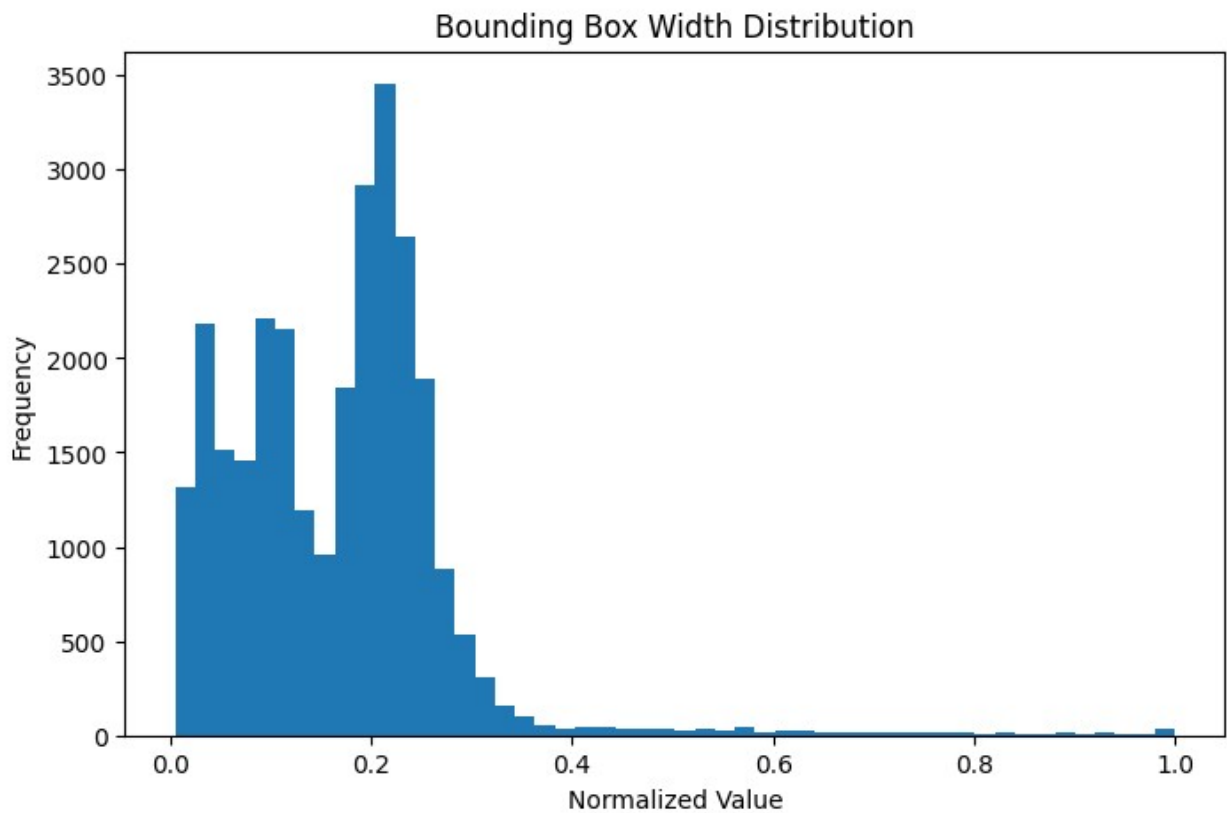
```

def plot_bbox_distribution(values, title):
    plt.figure(figsize=(8,5))
    plt.hist(values, bins=50)
    plt.title(title)
    plt.xlabel("Normalized Value")
    plt.ylabel("Frequency")
    plt.show()

plot_bbox_distribution(
    widths,
    "Bounding Box Width Distribution"
)

plot_bbox_distribution(
    heights,
    "Bounding Box Height Distribution"
)

```



Bounding Box Aspect Ratio Analysis

Purpose

License plates are rectangular objects.

Aspect ratio analysis helps:

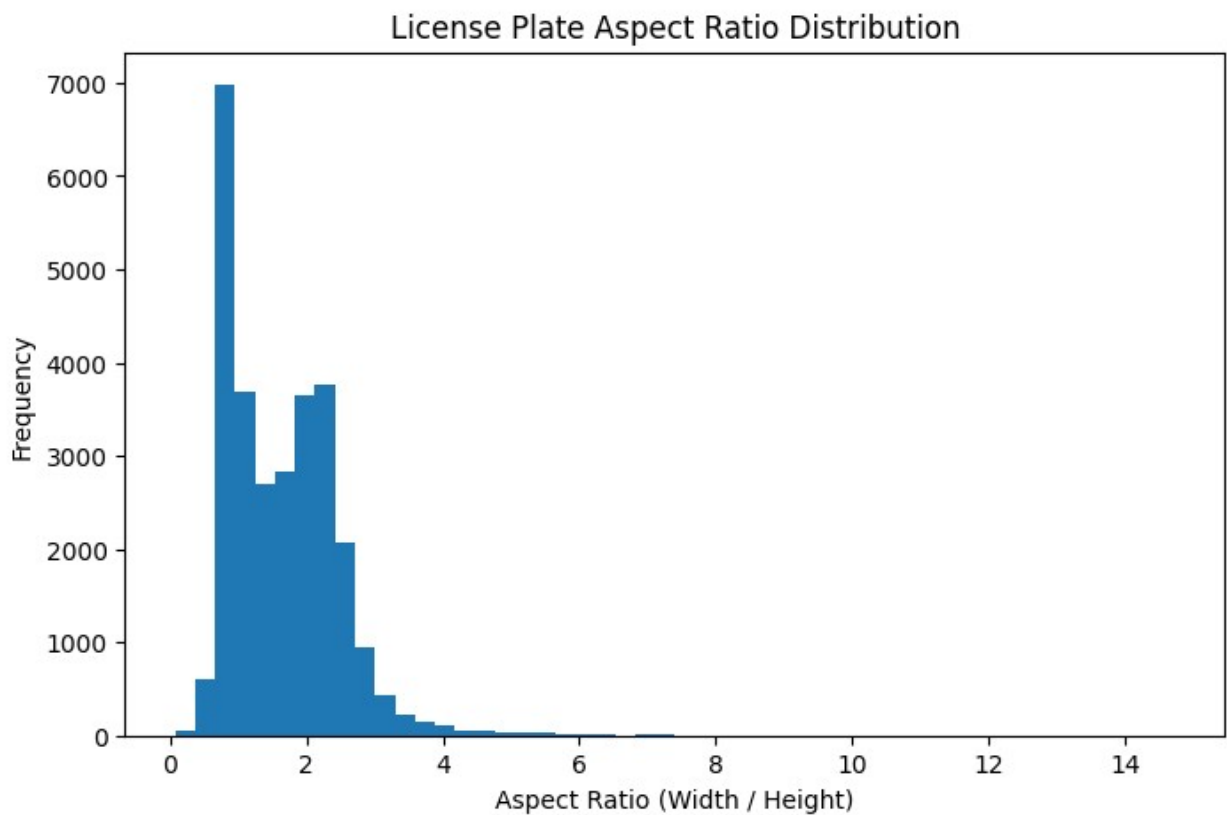
- understand object geometry,
- justify model choice,
- validate annotations.

```
def analyze_aspect_ratios(label_dir):
    aspect_ratios = []
    for file in os.listdir(label_dir):
        label_path = os.path.join(
            label_dir,
            file
        )
        with open(label_path, "r") as f:
            lines = f.readlines()
            for line in lines:
                _, _, _, w, h = map(
                    float,
                    line.strip().split()
                )
                if h > 0:
                    aspect_ratio = w / h
                    aspect_ratios.append(aspect_ratio)
    return aspect_ratios
```

Plot Aspect Ratios

```
aspect_ratios = analyze_aspect_ratios(
    paths["train_labels"]
)
plt.figure(figsize=(8,5))
plt.hist(aspect_ratios, bins=50)
```

```
plt.title("License Plate Aspect Ratio Distribution")
plt.xlabel("Aspect Ratio (Width / Height)")
plt.ylabel("Frequency")
plt.show()
```



Number of Annotations Per Image

Purpose

Checks:

- annotation density,
- missing labels,
- multi-object scenarios.

```
def annotations_per_image(label_dir):
    annotation_counts = []
    for file in os.listdir(label_dir):
        label_path = os.path.join(
```

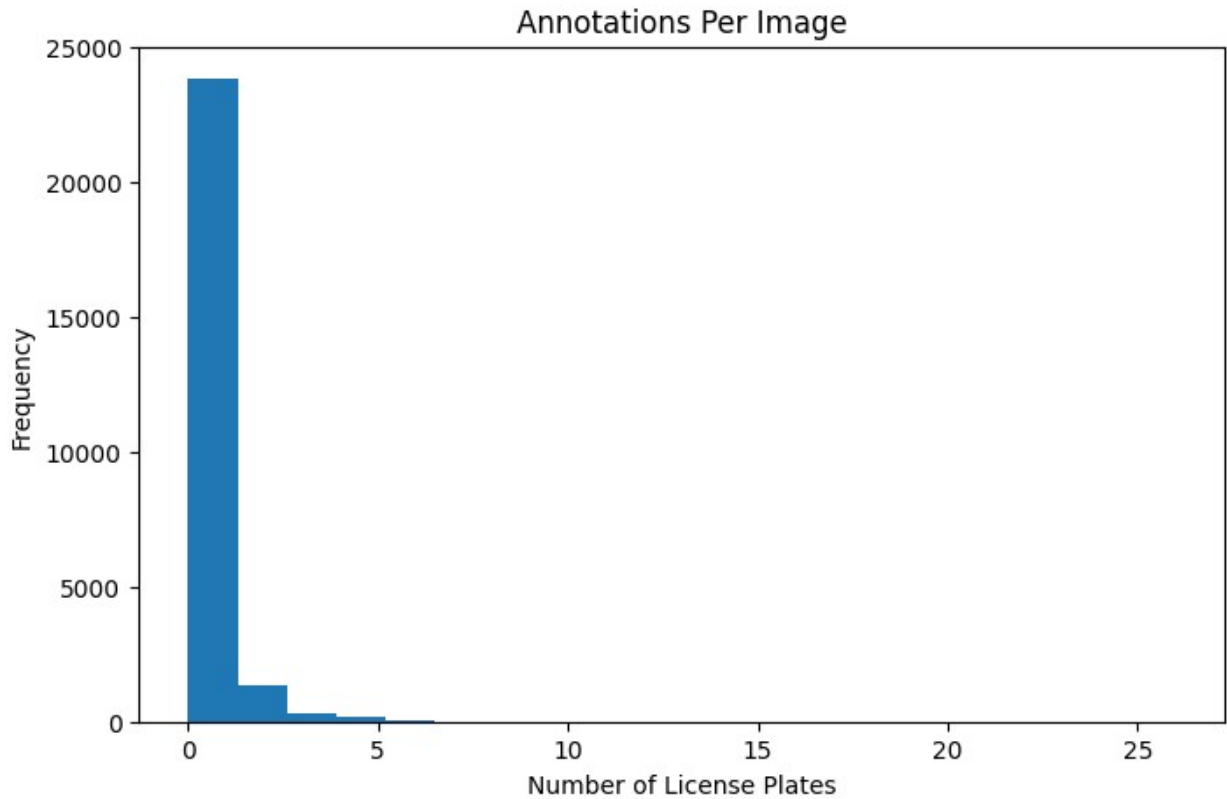
```
        label_dir,
        file
    )

    with open(label_path, "r") as f:
        lines = f.readlines()
        annotation_counts.append(len(lines))

    return annotation_counts

annotation_counts = annotations_per_image(
    paths["train_labels"]
)

plt.figure(figsize=(8,5))
plt.hist(annotation_counts, bins=20)
plt.title("Annotations Per Image")
plt.xlabel("Number of License Plates")
plt.ylabel("Frequency")
plt.show()
```



EDA Insights Section

EDA Insights

Key observations from the dataset analysis:

1. License plates exhibit strong rectangular aspect ratios.
2. Most plates occupy a relatively small portion of the image.
3. Image dimensions vary significantly, validating the need for resizing and normalization.
4. Most images contain a single license plate annotation.
5. Small-object characteristics justified the selection of YOLOv8m for improved feature extraction.

These findings directly influenced:

- model selection,
- input resolution,
- augmentation strategy,
- blur kernel design.

SECTION 8 — Coordinate Transformation

Coordinate Transformation

YOLO stores normalized coordinates.

OpenCV requires pixel coordinates.

This transformation converts:

- center_x,
- center_y,
- width,
- height

into:

- x1,
- y1,
- x2,
- y2

YOLO to Pixel Conversion

```
def yolo_to_pixel(box, img_width, img_height):  
    center_x, center_y, width, height = box  
    x1 = int((center_x - width/2) * img_width)  
    y1 = int((center_y - height/2) * img_height)  
    x2 = int((center_x + width/2) * img_width)  
    y2 = int((center_y + height/2) * img_height)  
    return x1, y1, x2, y2
```

SECTION 9 — Bounding Box Visualization

Draw Bounding Boxes

```
def draw_bounding_boxes(image_path, label_path):  
    image = cv2.imread(image_path)  
    image = cv2.cvtColor(  
        image,
```

```

        cv2.COLOR_BGR2RGB
    )
    h, w, _ = image.shape
    with open(label_path, "r") as f:
        labels = f.readlines()
    for label in labels:
        cls, x, y, bw, bh = map(
            float,
            label.strip().split()
        )
        x1, y1, x2, y2 = yolo_to_pixel(
            (x,y,bw,bh),
            w,
            h
        )
        cv2.rectangle(
            image,
            (x1,y1),
            (x2,y2),
            (0,255,0),
            2
        )
    plt.figure(figsize=(12,8))
    plt.imshow(image)
    plt.axis("off")
    plt.show()

```

Visualize Random Annotation

```

def visualize_random_annotation(paths):
    sample_image = random.choice(
        os.listdir(paths["train_images"])
    )
    image_path = os.path.join(
        paths["train_images"],
        sample_image
    )

```

```
label_path = os.path.join(
    paths["train_labels"],
    sample_image.replace(".jpg", ".txt")
)

draw_bounding_boxes(
    image_path,
    label_path
)

visualize_random_annotation(paths)
```



SECTION 10 — YOLOv8 Training

Load Model

```
def load_model(model_name="yolov8m.pt"):
    model = YOLO(model_name)
    return model

model = load_model()

/ultralytics/assets/releases/download/v8.4.0/yolov8m.pt to
'yolov8m.pt': 100% _____ 49.7MB 212.5MB/s 0.2s0.2s<0.0s
```

Train Model

```
def train_model(model, yaml_path):
    results = model.train(
        data=yaml_path,
        epochs=10,
        imgsz=640,
        batch=16,
        device=0,
        cache=True,
        amp=True,
        workers=8,
        freeze=5,
        augment=False,
        mosaic=0,
        mixup=0
    )
    return results

train_results = train_model(
    model,
    yaml_path
)

Ultralytics 8.4.48 □ Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla
T4, 14913MiB)
```

```

engine/trainer: agnostic_nms=False, amp=True, angle=1.0,
augment=False, auto_augment=randaugument, batch=16, bgr=0.0, box=7.5,
cache=True, cfg=None, classes=None, close_mosaic=10, cls=0.5,
cls_pw=0.0, compile=False, conf=None, copy_paste=0.0,
copy_paste_mode=flip, cos_lr=False, cutmix=0.0,
data=/kaggle/working/data.yaml, degrees=0.0, deterministic=True,
device=0, dfl=1.5, dnn=False, dropout=0.0, dynamic=False, embed=None,
end2end=None, epochs=10, erasing=0.4, exist_ok=False, fliplr=0.5,
flipud=0.0, format=torchscript, fraction=1.0, freeze=5, half=False,
hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, imgsz=640, int8=False, iou=0.7,
keras=False, kobj=1.0, line_width=None, lr0=0.01, lrf=0.01,
mask_ratio=4, max_det=300, mixup=0, mode=train, model=yolov8m.pt,
momentum=0.937, mosaic=0, multi_scale=0.0, name=train, nbs=64,
nms=False, opset=None, optimize=False, optimizer=auto,
overlap_mask=True, patience=100, perspective=0.0, plots=True,
pose=12.0, pretrained=True, profile=False, project=None, rect=False,
resume=False, retina_masks=False, rle=1.0, save=True, save_conf=False,
save_crop=False, save_dir=/kaggle/working/runs/detect/train,
save_frames=False, save_json=False, save_period=-1, save_txt=False,
scale=0.5, seed=0, shear=0.0, show=False, show_boxes=True,
show_conf=True, show_labels=True, simplify=True, single_cls=False,
source=None, split=val, stream_buffer=False, task=detect, time=None,
tracker=botsort.yaml, translate=0.1, val=True, verbose=True,
vid_stride=1, visualize=False, warmup_bias_lr=0.1, warmup_epochs=3.0,
warmup_momentum=0.8, weight_decay=0.0005, workers=8, workspace=None
/assets/Arial.ttf to '/root/.config/Ultralytics/Arial.ttf': 100%

```

755.1KB 17.1MB/s 0.0s

Overriding model.yaml nc=80 with nc=1

	from	n	params	module
arguments				
0	-1	1	1392	ultralytics.nn.modules.conv.Conv
[3, 48, 3, 2]				
1	-1	1	41664	ultralytics.nn.modules.conv.Conv
[48, 96, 3, 2]				
2	-1	2	111360	ultralytics.nn.modules.block.C2f
[96, 96, 2, True]				
3	-1	1	166272	ultralytics.nn.modules.conv.Conv
[96, 192, 3, 2]				
4	-1	4	813312	ultralytics.nn.modules.block.C2f
[192, 192, 4, True]				
5	-1	1	664320	ultralytics.nn.modules.conv.Conv
[192, 384, 3, 2]				
6	-1	4	3248640	ultralytics.nn.modules.block.C2f
[384, 384, 4, True]				
7	-1	1	1991808	ultralytics.nn.modules.conv.Conv
[384, 576, 3, 2]				
8	-1	2	3985920	ultralytics.nn.modules.block.C2f
[576, 576, 2, True]				

9	-1 1	831168	
ultralytics.nn.modules.block.SPPF			[576, 576, 5]
10	-1 1	0	
torch.nn.modules.upsampling.Upsample			[None, 2, 'nearest']
11	[-1, 6] 1	0	
ultralytics.nn.modules.conv.Concat			[1]
12	-1 2	1993728	ultralytics.nn.modules.block.C2f
[960, 384, 2]			
13	-1 1	0	
torch.nn.modules.upsampling.Upsample			[None, 2, 'nearest']
14	[-1, 4] 1	0	
ultralytics.nn.modules.conv.Concat			[1]
15	-1 2	517632	ultralytics.nn.modules.block.C2f
[576, 192, 2]			
16	-1 1	332160	ultralytics.nn.modules.conv.Conv
[192, 192, 3, 2]			
17	[-1, 12] 1	0	
ultralytics.nn.modules.conv.Concat			[1]
18	-1 2	1846272	ultralytics.nn.modules.block.C2f
[576, 384, 2]			
19	-1 1	1327872	ultralytics.nn.modules.conv.Conv
[384, 384, 3, 2]			
20	[-1, 9] 1	0	
ultralytics.nn.modules.conv.Concat			[1]
21	-1 2	4207104	ultralytics.nn.modules.block.C2f
[960, 576, 2]			
22	[15, 18, 21] 1	3776275	
ultralytics.nn.modules.head.Detect			[1, 16, None, [192, 384, 576]]

Model summary: 170 layers, 25,856,899 parameters, 25,856,883 gradients, 79.1 GFLOPs

Transferred 469/475 items from pretrained weights

Freezing layer 'model.0.conv.weight'
 Freezing layer 'model.0.bn.weight'
 Freezing layer 'model.0.bn.bias'
 Freezing layer 'model.1.conv.weight'
 Freezing layer 'model.1.bn.weight'
 Freezing layer 'model.1.bn.bias'
 Freezing layer 'model.2.cv1.conv.weight'
 Freezing layer 'model.2.cv1.bn.weight'
 Freezing layer 'model.2.cv1.bn.bias'
 Freezing layer 'model.2.cv2.conv.weight'

```
Freezing layer 'model.2.cv2.bn.weight'
Freezing layer 'model.2.cv2.bn.bias'
Freezing layer 'model.2.m.0.cv1.conv.weight'
Freezing layer 'model.2.m.0.cv1.bn.weight'
Freezing layer 'model.2.m.0.cv1.bn.bias'
Freezing layer 'model.2.m.0.cv2.conv.weight'
Freezing layer 'model.2.m.0.cv2.bn.weight'
Freezing layer 'model.2.m.0.cv2.bn.bias'
Freezing layer 'model.2.m.1.cv1.conv.weight'
Freezing layer 'model.2.m.1.cv1.bn.weight'
Freezing layer 'model.2.m.1.cv1.bn.bias'
Freezing layer 'model.2.m.1.cv2.conv.weight'
Freezing layer 'model.2.m.1.cv2.bn.weight'
Freezing layer 'model.2.m.1.cv2.bn.bias'
Freezing layer 'model.3.conv.weight'
Freezing layer 'model.3.bn.weight'
Freezing layer 'model.3.bn.bias'
Freezing layer 'model.4.cv1.conv.weight'
Freezing layer 'model.4.cv1.bn.weight'
Freezing layer 'model.4.cv1.bn.bias'
Freezing layer 'model.4.cv2.conv.weight'
Freezing layer 'model.4.cv2.bn.weight'
Freezing layer 'model.4.cv2.bn.bias'
Freezing layer 'model.4.m.0.cv1.conv.weight'
Freezing layer 'model.4.m.0.cv1.bn.weight'
Freezing layer 'model.4.m.0.cv1.bn.bias'
Freezing layer 'model.4.m.0.cv2.conv.weight'
Freezing layer 'model.4.m.0.cv2.bn.weight'
Freezing layer 'model.4.m.0.cv2.bn.bias'
Freezing layer 'model.4.m.1.cv1.conv.weight'
Freezing layer 'model.4.m.1.cv1.bn.weight'
Freezing layer 'model.4.m.1.cv1.bn.bias'
Freezing layer 'model.4.m.1.cv2.conv.weight'
Freezing layer 'model.4.m.1.cv2.bn.weight'
Freezing layer 'model.4.m.1.cv2.bn.bias'
Freezing layer 'model.4.m.2.cv1.conv.weight'
Freezing layer 'model.4.m.2.cv1.bn.weight'
Freezing layer 'model.4.m.2.cv1.bn.bias'
Freezing layer 'model.4.m.2.cv2.conv.weight'
Freezing layer 'model.4.m.2.cv2.bn.weight'
Freezing layer 'model.4.m.2.cv2.bn.bias'
Freezing layer 'model.4.m.3.cv1.conv.weight'
Freezing layer 'model.4.m.3.cv1.bn.weight'
Freezing layer 'model.4.m.3.cv1.bn.bias'
Freezing layer 'model.4.m.3.cv2.conv.weight'
Freezing layer 'model.4.m.3.cv2.bn.weight'
Freezing layer 'model.4.m.3.cv2.bn.bias'
Freezing layer 'model.22.dfl.conv.weight'
AMP: running Automatic Mixed Precision (AMP) checks...
```

```
/ultralytics/assets/releases/download/v8.4.0/yolo26n.pt to
'yolo26n.pt': 100% _____ 5.3MB 64.7MB/s 0.1s
AMP: checks passed ☐
train: Fast image access ☐ (ping: 0.8±0.7 ms, read: 10.4±11.5 MB/s,
size: 76.5 KB)
train: Scanning /kaggle/input/datasets/gangadharpanda/license-plate-
detection-and-blurring/License Plate Detection and
Blurring/labels/train... 25470 images, 173 backgrounds, 0 corrupt:
100% _____ 25625/25625 402.3it/s 1:04<0.0ss
WARNING ⚠ train: Cache directory
/kaggle/input/datasets/gangadharpanda/license-plate-detection-and-
blurring/License Plate Detection and Blurring/labels is not writable,
cache not saved.
WARNING ⚠ train: 41.4GB RAM required to cache images with 50% safety
margin but only 28.6/31.4GB available, not caching images
albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01,
blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average',
num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0),
tile_grid_size=(8, 8))
val: Fast image access ☐ (ping: 0.0±0.0 ms, read: 25.9±7.9 MB/s, size:
148.5 KB)
val: Scanning /kaggle/input/datasets/gangadharpanda/license-plate-
detection-and-blurring/License Plate Detection and
Blurring/labels/val... 1073 images, 0 backgrounds, 0 corrupt: 100%
_____ 1073/1073 246.3it/s 4.4s0.1s
WARNING ⚠ val: Cache directory
/kaggle/input/datasets/gangadharpanda/license-plate-detection-and-
blurring/License Plate Detection and Blurring/labels is not writable,
cache not saved.
WARNING ⚠ cache='ram' may produce non-deterministic training results.
Consider cache='disk' as a deterministic alternative if your disk
space allows.
val: Caching images (0.9GB RAM): 100% _____ 1073/1073 274.4it/s
3.9s0.1s
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and
'momentum=0.937' and determining best 'optimizer', 'lr0' and
'momentum' automatically...
optimizer: AdamW(lr=0.002, momentum=0.9) with parameter groups 77
weight(decay=0.0), 84 weight(decay=0.0005), 83 bias(decay=0.0)
Plotting labels to /kaggle/working/runs/detect/train/labels.jpg...
Image sizes 640 train, 640 val
Using 2 dataloader workers
Logging results to /kaggle/working/runs/detect/train
Starting training for 10 epochs...
Closing dataloader mosaic
albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01,
blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average',
num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0),
tile_grid_size=(8, 8))
```

Epoch	GPU_mem	box_loss	cls_loss	df_l_loss	Instances
Size					
ages	Instances	Box(P	R	mAP50	mAP50-95): 100%
	34/34	2.8it/s	12.0s0.4s		
		all	1073	1573	0.629 0.631
0.598	0.269				

Epoch	GPU_mem	box_loss	cls_loss	df_l_loss	Instances
Size					
ages	Instances	Box(P	R	mAP50	mAP50-95): 100%
	34/34	3.0it/s	11.2s0.4s		
		all	1073	1573	0.751 0.688
0.706	0.313				

Epoch	GPU_mem	box_loss	cls_loss	df_l_loss	Instances
Size					
ages	Instances	Box(P	R	mAP50	mAP50-95): 100%
	34/34	3.0it/s	11.2s0.4s		
		all	1073	1573	0.758 0.719
0.72	0.339				

Epoch	GPU_mem	box_loss	cls_loss	df_l_loss	Instances
Size					
ages	Instances	Box(P	R	mAP50	mAP50-95): 100%
	34/34	3.0it/s	11.2s0.4s		
		all	1073	1573	0.809 0.736
0.767	0.358				

Epoch	GPU_mem	box_loss	cls_loss	df_l_loss	Instances
Size					
ages	Instances	Box(P	R	mAP50	mAP50-95): 100%
	34/34	3.1it/s	11.1s0.4s		
		all	1073	1573	0.825 0.762
0.806	0.381				

Epoch	GPU_mem	box_loss	cls_loss	df_l_loss	Instances
Size					
ages	Instances	Box(P	R	mAP50	mAP50-95): 100%
	34/34	3.0it/s	11.2s0.4s		
		all	1073	1573	0.811 0.781
0.814	0.405				

Epoch	GPU_mem	box_loss	cls_loss	df_l_loss	Instances
Size					
ages	Instances	Box(P	R	mAP50	mAP50-95): 100%
	34/34	3.0it/s	11.2s0.4s		
		all	1073	1573	0.814 0.782
0.818	0.408				

Epoch	GPU_mem	box_loss	cls_loss	df_l_loss	Instances
Size					
ages	Instances	Box(P	R	mAP50	mAP50-95): 100%
34/34	3.0it/s	11.2s0.4s			
	all	1073	1573	0.833	0.81
0.839	0.415				

Epoch	GPU_mem	box_loss	cls_loss	df_l_loss	Instances
Size					
ages	Instances	Box(P	R	mAP50	mAP50-95): 100%
34/34	3.0it/s	11.4s0.4s			
	all	1073	1573	0.843	0.803
0.847	0.427				

Epoch	GPU_mem	box_loss	cls_loss	df_l_loss	Instances
Size					
ages	Instances	Box(P	R	mAP50	mAP50-95): 100%
34/34	3.0it/s	11.3s0.4s			
	all	1073	1573	0.848	0.8
0.853	0.433				

10 epochs completed in 1.733 hours.

Optimizer stripped from

/kaggle/working/runs/detect/train/weights/last.pt, 52.0MB

Optimizer stripped from

/kaggle/working/runs/detect/train/weights/best.pt, 52.0MB

Validating /kaggle/working/runs/detect/train/weights/best.pt...

Ultralytics 8.4.48 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

Model summary (fused): 93 layers, 25,840,339 parameters, 0 gradients, 78.7 GFLOPs

Epoch	GPU_mem	box_loss	cls_loss	df_l_loss	Instances
Size					
ages	Instances	Box(P	R	mAP50	mAP50-95): 100%
34/34	2.8it/s	12.3s0.4s			
	all	1073	1573	0.849	0.8
0.853	0.433				

Speed: 0.1ms preprocess, 7.7ms inference, 0.0ms loss, 0.9ms postprocess per image

Results saved to /kaggle/working/runs/detect/train

SECTION 11 — Model Evaluation

Validate Model

```
def validate_model(model):
    metrics = model.val()
    return metrics
```

```
metrics = validate_model(model)
```

```
print(metrics)
```

```
Ultralytics 8.4.48 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
```

```
Model summary (fused): 93 layers, 25,840,339 parameters, 0 gradients, 78.7 GFLOPs
```

```
val: Fast image access (ping: 0.0±0.0 ms, read: 520.8±372.2 MB/s, size: 374.9 KB)
```

```
val: Scanning /kaggle/input/datasets/gangadharpanda/license-plate-detection-and-blurring/License Plate Detection and Blurring/labels/val... 1073 images, 0 backgrounds, 0 corrupt: 100%  
1073/1073 1.0Kit/s 1.0s0.0s
```

```
WARNING ⚠ val: Cache directory /kaggle/input/datasets/gangadharpanda/license-plate-detection-and-blurring/License Plate Detection and Blurring/labels is not writable, cache not saved.
```

ages	Instances	Box(P	R	mAP50	mAP50-95): 100%
	68/68	3.0it/s	22.9s0.3ss		
		all	1073	1573	0.851 0.803

```
0.855 0.432
```

```
Speed: 0.7ms preprocess, 17.2ms inference, 0.0ms loss, 0.7ms postprocess per image
```

```
Results saved to /kaggle/working/runs/detect/val
```

```
ultralytics.utils.metrics.DetMetrics object with attributes:
```

```
ap_class_index: array([0])
```

```
box: ultralytics.utils.metrics.Metric object
```

```
confusion_matrix: <ultralytics.utils.metrics.ConfusionMatrix object at 0x7b26a95e9520>
```

```
curves: ['Precision-Recall(B)', 'F1-Confidence(B)', 'Precision-Confidence(B)', 'Recall-Confidence(B)']
```

```
curves_results: [[array([0, 0.001001, 0.002002, 0.003003, 0.004004, 0.005005, 0.006006, 0.007007, 0.008008, 0.009009, 0.01001, 0.011011, 0.012012, 0.013013, 0.014014, 0.015015, 0.016016, 0.017017, 0.018018, 0.019019, 0.02002, 0.021021, 0.022022, 0.023023, 0.024024, 0.025025, 0.026026, 0.027027, 0.028028, 0.029029, 0.03003, 0.031031, 0.032032, 0.033033, 0.034034, 0.035035, 0.036036, 0.037037, 0.038038, 0.039039, 0.04004, 0.041041, 0.042042, 0.043043, 0.044044, 0.045045, 0.046046, 0.047047, 0.048048, 0.049049, 0.05005, 0.051051, 0.052052, 0.053053, 0.054054, 0.055055, 0.056056, 0.057057, 0.058058, 0.059059, 0.06006, 0.061061, 0.062062, 0.063063, 0.064064, 0.065065, 0.066066, 0.067067, 0.068068, 0.069069, 0.07007, 0.071071,
```

	0.072072,	0.073073,	0.074074,	0.075075,
0.076076,	0.077077,	0.078078,	0.079079,	0.08008,
0.081081,	0.082082,	0.083083,	0.084084,	0.085085,
0.086086,	0.087087,	0.088088,	0.089089,	0.09009,
0.091091,	0.092092,	0.093093,	0.094094,	0.095095,
	0.096096,	0.097097,	0.098098,	0.099099,
0.1001,	0.1011,	0.1021,	0.1031,	0.1041,
0.10511,	0.10611,	0.10711,	0.10811,	0.10911,
0.11011,	0.11111,	0.11211,	0.11311,	0.11411,
0.11512,	0.11612,	0.11712,	0.11812,	0.11912,
	0.12012,	0.12112,	0.12212,	0.12312,
0.12412,	0.12513,	0.12613,	0.12713,	0.12813,
0.12913,	0.13013,	0.13113,	0.13213,	0.13313,
0.13413,	0.13514,	0.13614,	0.13714,	0.13814,
0.13914,	0.14014,	0.14114,	0.14214,	0.14314,
	0.14414,	0.14515,	0.14615,	0.14715,
0.14815,	0.14915,	0.15015,	0.15115,	0.15215,
0.15315,	0.15415,	0.15516,	0.15616,	0.15716,
0.15816,	0.15916,	0.16016,	0.16116,	0.16216,
0.16316,	0.16416,	0.16517,	0.16617,	0.16717,
	0.16817,	0.16917,	0.17017,	0.17117,
0.17217,	0.17317,	0.17417,	0.17518,	0.17618,
0.17718,	0.17818,	0.17918,	0.18018,	0.18118,
0.18218,	0.18318,	0.18418,	0.18519,	0.18619,
0.18719,	0.18819,	0.18919,	0.19019,	0.19119,
	0.19219,	0.19319,	0.19419,	0.1952,
0.1962,	0.1972,	0.1982,	0.1992,	0.2002,
0.2012,	0.2022,	0.2032,	0.2042,	0.20521,
0.20621,	0.20721,	0.20821,	0.20921,	0.21021,
0.21121,	0.21221,	0.21321,	0.21421,	0.21522,
	0.21622,	0.21722,	0.21822,	0.21922,
0.22022,	0.22122,	0.22222,	0.22322,	0.22422,
0.22523,	0.22623,	0.22723,	0.22823,	0.22923,
0.23023,	0.23123,	0.23223,	0.23323,	0.23423,
0.23524,	0.23624,	0.23724,	0.23824,	0.23924,
	0.24024,	0.24124,	0.24224,	0.24324,
0.24424,	0.24525,	0.24625,	0.24725,	0.24825,
0.24925,	0.25025,	0.25125,	0.25225,	0.25325,
0.25425,	0.25526,	0.25626,	0.25726,	0.25826,
0.25926,	0.26026,	0.26126,	0.26226,	0.26326,
	0.26426,	0.26527,	0.26627,	0.26727,
0.26827,	0.26927,	0.27027,	0.27127,	0.27227,
0.27327,	0.27427,	0.27528,	0.27628,	0.27728,
0.27828,	0.27928,	0.28028,	0.28128,	0.28228,
0.28328,	0.28428,	0.28529,	0.28629,	0.28729,
	0.28829,	0.28929,	0.29029,	0.29129,
0.29229,	0.29329,	0.29429,	0.2953,	0.2963,
0.2973,	0.2983,	0.2993,	0.3003,	0.3013,
0.3023,	0.3033,	0.3043,	0.30531,	0.30631,

0.30731,	0.30831,	0.30931,	0.31031,	0.31131,
	0.31231,	0.31331,	0.31431,	0.31532,
0.31632,	0.31732,	0.31832,	0.31932,	0.32032,
0.32132,	0.32232,	0.32332,	0.32432,	0.32533,
0.32633,	0.32733,	0.32833,	0.32933,	0.33033,
0.33133,	0.33233,	0.33333,	0.33433,	0.33534,
	0.33634,	0.33734,	0.33834,	0.33934,
0.34034,	0.34134,	0.34234,	0.34334,	0.34434,
0.34535,	0.34635,	0.34735,	0.34835,	0.34935,
0.35035,	0.35135,	0.35235,	0.35335,	0.35435,
0.35536,	0.35636,	0.35736,	0.35836,	0.35936,
	0.36036,	0.36136,	0.36236,	0.36336,
0.36436,	0.36537,	0.36637,	0.36737,	0.36837,
0.36937,	0.37037,	0.37137,	0.37237,	0.37337,
0.37437,	0.37538,	0.37638,	0.37738,	0.37838,
0.37938,	0.38038,	0.38138,	0.38238,	0.38338,
	0.38438,	0.38539,	0.38639,	0.38739,
0.38839,	0.38939,	0.39039,	0.39139,	0.39239,
0.39339,	0.39439,	0.3954,	0.3964,	0.3974,
0.3984,	0.3994,	0.4004,	0.4014,	0.4024,
0.4034,	0.4044,	0.40541,	0.40641,	0.40741,
	0.40841,	0.40941,	0.41041,	0.41141,
0.41241,	0.41341,	0.41441,	0.41542,	0.41642,
0.41742,	0.41842,	0.41942,	0.42042,	0.42142,
0.42242,	0.42342,	0.42442,	0.42543,	0.42643,
0.42743,	0.42843,	0.42943,	0.43043,	0.43143,
	0.43243,	0.43343,	0.43443,	0.43544,
0.43644,	0.43744,	0.43844,	0.43944,	0.44044,
0.44144,	0.44244,	0.44344,	0.44444,	0.44545,
0.44645,	0.44745,	0.44845,	0.44945,	0.45045,
0.45145,	0.45245,	0.45345,	0.45445,	0.45546,
	0.45646,	0.45746,	0.45846,	0.45946,
0.46046,	0.46146,	0.46246,	0.46346,	0.46446,
0.46547,	0.46647,	0.46747,	0.46847,	0.46947,
0.47047,	0.47147,	0.47247,	0.47347,	0.47447,
0.47548,	0.47648,	0.47748,	0.47848,	0.47948,
	0.48048,	0.48148,	0.48248,	0.48348,
0.48448,	0.48549,	0.48649,	0.48749,	0.48849,
0.48949,	0.49049,	0.49149,	0.49249,	0.49349,
0.49449,	0.4955,	0.4965,	0.4975,	0.4985,
0.4995,	0.5005,	0.5015,	0.5025,	0.5035,
	0.5045,	0.50551,	0.50651,	0.50751,
0.50851,	0.50951,	0.51051,	0.51151,	0.51251,
0.51351,	0.51451,	0.51552,	0.51652,	0.51752,
0.51852,	0.51952,	0.52052,	0.52152,	0.52252,
0.52352,	0.52452,	0.52553,	0.52653,	0.52753,
	0.52853,	0.52953,	0.53053,	0.53153,
0.53253,	0.53353,	0.53453,	0.53554,	0.53654,
0.53754,	0.53854,	0.53954,	0.54054,	0.54154,

0.54254,	0.54354,	0.54454,	0.54555,	0.54655,
0.54755,	0.54855,	0.54955,	0.55055,	0.55155,
	0.55255,	0.55355,	0.55455,	0.55556,
0.55656,	0.55756,	0.55856,	0.55956,	0.56056,
0.56156,	0.56256,	0.56356,	0.56456,	0.56557,
0.56657,	0.56757,	0.56857,	0.56957,	0.57057,
0.57157,	0.57257,	0.57357,	0.57457,	0.57558,
	0.57658,	0.57758,	0.57858,	0.57958,
0.58058,	0.58158,	0.58258,	0.58358,	0.58458,
0.58559,	0.58659,	0.58759,	0.58859,	0.58959,
0.59059,	0.59159,	0.59259,	0.59359,	0.59459,
0.5956,	0.5966,	0.5976,	0.5986,	0.5996,
	0.6006,	0.6016,	0.6026,	0.6036,
0.6046,	0.60561,	0.60661,	0.60761,	0.60861,
0.60961,	0.61061,	0.61161,	0.61261,	0.61361,
0.61461,	0.61562,	0.61662,	0.61762,	0.61862,
0.61962,	0.62062,	0.62162,	0.62262,	0.62362,
	0.62462,	0.62563,	0.62663,	0.62763,
0.62863,	0.62963,	0.63063,	0.63163,	0.63263,
0.63363,	0.63463,	0.63564,	0.63664,	0.63764,
0.63864,	0.63964,	0.64064,	0.64164,	0.64264,
0.64364,	0.64464,	0.64565,	0.64665,	0.64765,
	0.64865,	0.64965,	0.65065,	0.65165,
0.65265,	0.65365,	0.65465,	0.65566,	0.65666,
0.65766,	0.65866,	0.65966,	0.66066,	0.66166,
0.66266,	0.66366,	0.66466,	0.66567,	0.66667,
0.66767,	0.66867,	0.66967,	0.67067,	0.67167,
	0.67267,	0.67367,	0.67467,	0.67568,
0.67668,	0.67768,	0.67868,	0.67968,	0.68068,
0.68168,	0.68268,	0.68368,	0.68468,	0.68569,
0.68669,	0.68769,	0.68869,	0.68969,	0.69069,
0.69169,	0.69269,	0.69369,	0.69469,	0.6957,
	0.6967,	0.6977,	0.6987,	0.6997,
0.7007,	0.7017,	0.7027,	0.7037,	0.7047,
0.70571,	0.70671,	0.70771,	0.70871,	0.70971,
0.71071,	0.71171,	0.71271,	0.71371,	0.71471,
0.71572,	0.71672,	0.71772,	0.71872,	0.71972,
	0.72072,	0.72172,	0.72272,	0.72372,
0.72472,	0.72573,	0.72673,	0.72773,	0.72873,
0.72973,	0.73073,	0.73173,	0.73273,	0.73373,
0.73473,	0.73574,	0.73674,	0.73774,	0.73874,
0.73974,	0.74074,	0.74174,	0.74274,	0.74374,
	0.74474,	0.74575,	0.74675,	0.74775,
0.74875,	0.74975,	0.75075,	0.75175,	0.75275,
0.75375,	0.75475,	0.75576,	0.75676,	0.75776,
0.75876,	0.75976,	0.76076,	0.76176,	0.76276,
0.76376,	0.76476,	0.76577,	0.76677,	0.76777,
	0.76877,	0.76977,	0.77077,	0.77177,
0.77277,	0.77377,	0.77477,	0.77578,	0.77678,

0.77778,	0.77878,	0.77978,	0.78078,	0.78178,	
0.78278,	0.78378,	0.78478,	0.78579,	0.78679,	
0.78779,	0.78879,	0.78979,	0.79079,	0.79179,	
	0.79279,	0.79379,	0.79479,	0.7958,	
0.7968,	0.7978,	0.7988,	0.7998,	0.8008,	
0.8018,	0.8028,	0.8038,	0.8048,	0.80581,	
0.80681,	0.80781,	0.80881,	0.80981,	0.81081,	
0.81181,	0.81281,	0.81381,	0.81481,	0.81582,	
	0.81682,	0.81782,	0.81882,	0.81982,	
0.82082,	0.82182,	0.82282,	0.82382,	0.82482,	
0.82583,	0.82683,	0.82783,	0.82883,	0.82983,	
0.83083,	0.83183,	0.83283,	0.83383,	0.83483,	
0.83584,	0.83684,	0.83784,	0.83884,	0.83984,	
	0.84084,	0.84184,	0.84284,	0.84384,	
0.84484,	0.84585,	0.84685,	0.84785,	0.84885,	
0.84985,	0.85085,	0.85185,	0.85285,	0.85385,	
0.85485,	0.85586,	0.85686,	0.85786,	0.85886,	
0.85986,	0.86086,	0.86186,	0.86286,	0.86386,	
	0.86486,	0.86587,	0.86687,	0.86787,	
0.86887,	0.86987,	0.87087,	0.87187,	0.87287,	
0.87387,	0.87487,	0.87588,	0.87688,	0.87788,	
0.87888,	0.87988,	0.88088,	0.88188,	0.88288,	
0.88388,	0.88488,	0.88589,	0.88689,	0.88789,	
	0.88889,	0.88989,	0.89089,	0.89189,	
0.89289,	0.89389,	0.89489,	0.8959,	0.8969,	
0.8979,	0.8989,	0.8999,	0.9009,	0.9019,	
0.9029,	0.9039,	0.9049,	0.90591,	0.90691,	
0.90791,	0.90891,	0.90991,	0.91091,	0.91191,	
	0.91291,	0.91391,	0.91491,	0.91592,	
0.91692,	0.91792,	0.91892,	0.91992,	0.92092,	
0.92192,	0.92292,	0.92392,	0.92492,	0.92593,	
0.92693,	0.92793,	0.92893,	0.92993,	0.93093,	
0.93193,	0.93293,	0.93393,	0.93493,	0.93594,	
	0.93694,	0.93794,	0.93894,	0.93994,	
0.94094,	0.94194,	0.94294,	0.94394,	0.94494,	
0.94595,	0.94695,	0.94795,	0.94895,	0.94995,	
0.95095,	0.95195,	0.95295,	0.95395,	0.95495,	
0.95596,	0.95696,	0.95796,	0.95896,	0.95996,	
	0.96096,	0.96196,	0.96296,	0.96396,	
0.96496,	0.96597,	0.96697,	0.96797,	0.96897,	
0.96997,	0.97097,	0.97197,	0.97297,	0.97397,	
0.97497,	0.97598,	0.97698,	0.97798,	0.97898,	
0.97998,	0.98098,	0.98198,	0.98298,	0.98398,	
	0.98498,	0.98599,	0.98699,	0.98799,	
0.98899,	0.98999,	0.99099,	0.99199,	0.99299,	
0.99399,	0.99499,	0.996,	0.997,	0.998,	
0.999,	1]],	array([[	1,	1,	1,
1,	1,	1,	1,	1,	1,
1,	1,	1,	1,	1,	1,

[illegible]

[illegible]

[illegible]

	0.92353,	0.92295,	0.92295,	0.92172,
0.92172,	0.92172,	0.9203,	0.91967,	0.91903,
0.91903,	0.91729,	0.91729,	0.91729,	0.91729,
0.91729,	0.91701,	0.91701,	0.91701,	0.91701,
0.91564,	0.91564,	0.91517,	0.91517,	
	0.9147,	0.9147,	0.9147,	0.91431,
0.91431,	0.91431,	0.91004,	0.90953,	0.90953,
0.90902,	0.90902,	0.90851,	0.90851,	0.90585,
0.90585,	0.90315,	0.90259,	0.90141,	0.90141,
0.90016,	0.89984,	0.89984,	0.89984,	
	0.89984,	0.89799,	0.89684,	0.89684,
0.89623,	0.89563,	0.8945,	0.8945,	0.89278,
0.89278,	0.89183,	0.89183,	0.89183,	0.8914,
0.8914,	0.8909,	0.89039,	0.89039,	0.88856,
0.88798,	0.88658,	0.88658,	0.88658,	
	0.88658,	0.88601,	0.88253,	0.88253,
0.88253,	0.88196,	0.88167,	0.88167,	0.88111,
0.87936,	0.87881,	0.87798,	0.87798,	0.87798,
0.87752,	0.87653,	0.87653,	0.87348,	0.87147,
0.87147,	0.87147,	0.87147,	0.87147,	
	0.87147,	0.87147,	0.87094,	0.87018,
0.87018,	0.87018,	0.86975,	0.86862,	0.86811,
0.86708,	0.86667,	0.86556,	0.8593,	0.8593,
0.85539,	0.85432,	0.85356,	0.85356,	0.85318,
0.85108,	0.85108,	0.8492,	0.8492,	
	0.8477,	0.84723,	0.84677,	0.84539,
0.84539,	0.84447,	0.84402,	0.84034,	0.8401,
0.8401,	0.83976,	0.83932,	0.83844,	0.83766,
0.83766,	0.83463,	0.83151,	0.82327,	0.82141,
0.81635,	0.81454,	0.81211,	0.80994,	
	0.80994,	0.80617,	0.80554,	0.80554,
0.8038,	0.80306,	0.80134,	0.80134,	0.79964,
0.7983,	0.79771,	0.79747,	0.79747,	0.79532,
0.79366,	0.79154,	0.79154,	0.78897,	0.78412,
0.78332,	0.77946,	0.77552,	0.77309,	
	0.77233,	0.77095,	0.77064,	0.77002,
0.76897,	0.76897,	0.76487,	0.7607,	0.75222,
0.74862,	0.74179,	0.73724,	0.73539,	0.73448,
0.7328,	0.72708,	0.72467,	0.7229,	0.72257,
0.72257,	0.70431,	0.70138,	0.69832,	
	0.69616,	0.69561,	0.69347,	0.69293,
0.68601,	0.68092,	0.67478,	0.66539,	0.66269,
0.64818,	0.64355,	0.6424,	0.63628,	0.63414,
0.63246,	0.6312,	0.62645,	0.62522,	0.61515,
0.59619,	0.59585,	0.59569,	0.59186,	
	0.58582,	0.58237,	0.57584,	0.56647,
0.56647,	0.55573,	0.55547,	0.5528,	0.54682,
0.53526,	0.52711,	0.52535,	0.5078,	0.50476,
0.49671,	0.47711,	0.47711,	0.46377,	0.45017,

0.4382,	0.43389,	0.42794,	0.41935,		
	0.41794,	0.41186,	0.39586,	0.36941,	
0.36826,	0.36136,	0.35603,	0.33781,	0.3049,	
0.2933,	0.24593,	0.23119,	0.22937,	0.17386,	
0.16957,	0.16365,	0.16349,	0.1554,	0.15219,	
0.14947,	0.12796,	0.12546,	0.11046,		
	0.096863,	0,	0,	0,	
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,					
	0,	0,	0,	0,	
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0]]), 'Recall', 'Precision'], [array([			0,	0.001001,	
0.002002,	0.003003,	0.004004,	0.005005,	0.006006,	
0.007007,	0.008008,	0.009009,	0.01001,	0.011011,	
0.012012,	0.013013,	0.014014,	0.015015,	0.016016,	
0.017017,	0.018018,	0.019019,	0.02002,	0.021021,	
0.022022,	0.023023,				
	0.024024,	0.025025,	0.026026,	0.027027,	
0.028028,	0.029029,	0.03003,	0.031031,	0.032032,	
0.033033,	0.034034,	0.035035,	0.036036,	0.037037,	
0.038038,	0.039039,	0.04004,	0.041041,	0.042042,	
0.043043,	0.044044,	0.045045,	0.046046,	0.047047,	
	0.048048,	0.049049,	0.05005,	0.051051,	
0.052052,	0.053053,	0.054054,	0.055055,	0.056056,	
0.057057,	0.058058,	0.059059,	0.06006,	0.061061,	
0.062062,	0.063063,	0.064064,	0.065065,	0.066066,	
0.067067,	0.068068,	0.069069,	0.07007,	0.071071,	
	0.072072,	0.073073,	0.074074,	0.075075,	
0.076076,	0.077077,	0.078078,	0.079079,	0.08008,	
0.081081,	0.082082,	0.083083,	0.084084,	0.085085,	
0.086086,	0.087087,	0.088088,	0.089089,	0.09009,	
0.091091,	0.092092,	0.093093,	0.094094,	0.095095,	
	0.096096,	0.097097,	0.098098,	0.099099,	
0.1001,	0.1011,	0.1021,	0.1031,	0.1041,	
0.10511,	0.10611,	0.10711,	0.10811,	0.10911,	
0.11011,	0.11111,	0.11211,	0.11311,	0.11411,	
0.11512,	0.11612,	0.11712,	0.11812,	0.11912,	
	0.12012,	0.12112,	0.12212,	0.12312,	
0.12412,	0.12513,	0.12613,	0.12713,	0.12813,	
0.12913,	0.13013,	0.13113,	0.13213,	0.13313,	
0.13413,	0.13514,	0.13614,	0.13714,	0.13814,	
0.13914,	0.14014,	0.14114,	0.14214,	0.14314,	

	0.14414,	0.14515,	0.14615,	0.14715,
0.14815,	0.14915,	0.15015,	0.15115,	0.15215,
0.15315,	0.15415,	0.15516,	0.15616,	0.15716,
0.15816,	0.15916,	0.16016,	0.16116,	0.16216,
0.16316,	0.16416,	0.16517,	0.16617,	0.16717,
	0.16817,	0.16917,	0.17017,	0.17117,
0.17217,	0.17317,	0.17417,	0.17518,	0.17618,
0.17718,	0.17818,	0.17918,	0.18018,	0.18118,
0.18218,	0.18318,	0.18418,	0.18519,	0.18619,
0.18719,	0.18819,	0.18919,	0.19019,	0.19119,
	0.19219,	0.19319,	0.19419,	0.1952,
0.1962,	0.1972,	0.1982,	0.1992,	0.2002,
0.2012,	0.2022,	0.2032,	0.2042,	0.20521,
0.20621,	0.20721,	0.20821,	0.20921,	0.21021,
0.21121,	0.21221,	0.21321,	0.21421,	0.21522,
	0.21622,	0.21722,	0.21822,	0.21922,
0.22022,	0.22122,	0.22222,	0.22322,	0.22422,
0.22523,	0.22623,	0.22723,	0.22823,	0.22923,
0.23023,	0.23123,	0.23223,	0.23323,	0.23423,
0.23524,	0.23624,	0.23724,	0.23824,	0.23924,
	0.24024,	0.24124,	0.24224,	0.24324,
0.24424,	0.24525,	0.24625,	0.24725,	0.24825,
0.24925,	0.25025,	0.25125,	0.25225,	0.25325,
0.25425,	0.25526,	0.25626,	0.25726,	0.25826,
0.25926,	0.26026,	0.26126,	0.26226,	0.26326,
	0.26426,	0.26527,	0.26627,	0.26727,
0.26827,	0.26927,	0.27027,	0.27127,	0.27227,
0.27327,	0.27427,	0.27528,	0.27628,	0.27728,
0.27828,	0.27928,	0.28028,	0.28128,	0.28228,
0.28328,	0.28428,	0.28529,	0.28629,	0.28729,
	0.28829,	0.28929,	0.29029,	0.29129,
0.29229,	0.29329,	0.29429,	0.2953,	0.2963,
0.2973,	0.2983,	0.2993,	0.3003,	0.3013,
0.3023,	0.3033,	0.3043,	0.30531,	0.30631,
0.30731,	0.30831,	0.30931,	0.31031,	0.31131,
	0.31231,	0.31331,	0.31431,	0.31532,
0.31632,	0.31732,	0.31832,	0.31932,	0.32032,
0.32132,	0.32232,	0.32332,	0.32432,	0.32533,
0.32633,	0.32733,	0.32833,	0.32933,	0.33033,
0.33133,	0.33233,	0.33333,	0.33433,	0.33534,
	0.33634,	0.33734,	0.33834,	0.33934,
0.34034,	0.34134,	0.34234,	0.34334,	0.34434,
0.34535,	0.34635,	0.34735,	0.34835,	0.34935,
0.35035,	0.35135,	0.35235,	0.35335,	0.35435,
0.35536,	0.35636,	0.35736,	0.35836,	0.35936,
	0.36036,	0.36136,	0.36236,	0.36336,
0.36436,	0.36537,	0.36637,	0.36737,	0.36837,
0.36937,	0.37037,	0.37137,	0.37237,	0.37337,
0.37437,	0.37538,	0.37638,	0.37738,	0.37838,
0.37938,	0.38038,	0.38138,	0.38238,	0.38338,

	0.38438,	0.38539,	0.38639,	0.38739,
0.38839,	0.38939,	0.39039,	0.39139,	0.39239,
0.39339,	0.39439,	0.3954,	0.3964,	0.3974,
0.3984,	0.3994,	0.4004,	0.4014,	0.4024,
0.4034,	0.4044,	0.40541,	0.40641,	0.40741,
	0.40841,	0.40941,	0.41041,	0.41141,
0.41241,	0.41341,	0.41441,	0.41542,	0.41642,
0.41742,	0.41842,	0.41942,	0.42042,	0.42142,
0.42242,	0.42342,	0.42442,	0.42543,	0.42643,
0.42743,	0.42843,	0.42943,	0.43043,	0.43143,
	0.43243,	0.43343,	0.43443,	0.43544,
0.43644,	0.43744,	0.43844,	0.43944,	0.44044,
0.44144,	0.44244,	0.44344,	0.44444,	0.44545,
0.44645,	0.44745,	0.44845,	0.44945,	0.45045,
0.45145,	0.45245,	0.45345,	0.45445,	0.45546,
	0.45646,	0.45746,	0.45846,	0.45946,
0.46046,	0.46146,	0.46246,	0.46346,	0.46446,
0.46547,	0.46647,	0.46747,	0.46847,	0.46947,
0.47047,	0.47147,	0.47247,	0.47347,	0.47447,
0.47548,	0.47648,	0.47748,	0.47848,	0.47948,
	0.48048,	0.48148,	0.48248,	0.48348,
0.48448,	0.48549,	0.48649,	0.48749,	0.48849,
0.48949,	0.49049,	0.49149,	0.49249,	0.49349,
0.49449,	0.4955,	0.4965,	0.4975,	0.4985,
0.4995,	0.5005,	0.5015,	0.5025,	0.5035,
	0.5045,	0.50551,	0.50651,	0.50751,
0.50851,	0.50951,	0.51051,	0.51151,	0.51251,
0.51351,	0.51451,	0.51552,	0.51652,	0.51752,
0.51852,	0.51952,	0.52052,	0.52152,	0.52252,
0.52352,	0.52452,	0.52553,	0.52653,	0.52753,
	0.52853,	0.52953,	0.53053,	0.53153,
0.53253,	0.53353,	0.53453,	0.53554,	0.53654,
0.53754,	0.53854,	0.53954,	0.54054,	0.54154,
0.54254,	0.54354,	0.54454,	0.54555,	0.54655,
0.54755,	0.54855,	0.54955,	0.55055,	0.55155,
	0.55255,	0.55355,	0.55455,	0.55556,
0.55656,	0.55756,	0.55856,	0.55956,	0.56056,
0.56156,	0.56256,	0.56356,	0.56456,	0.56557,
0.56657,	0.56757,	0.56857,	0.56957,	0.57057,
0.57157,	0.57257,	0.57357,	0.57457,	0.57558,
	0.57658,	0.57758,	0.57858,	0.57958,
0.58058,	0.58158,	0.58258,	0.58358,	0.58458,
0.58559,	0.58659,	0.58759,	0.58859,	0.58959,
0.59059,	0.59159,	0.59259,	0.59359,	0.59459,
0.5956,	0.5966,	0.5976,	0.5986,	0.5996,
	0.6006,	0.6016,	0.6026,	0.6036,
0.6046,	0.60561,	0.60661,	0.60761,	0.60861,
0.60961,	0.61061,	0.61161,	0.61261,	0.61361,
0.61461,	0.61562,	0.61662,	0.61762,	0.61862,

0.61962,	0.62062,	0.62162,	0.62262,	0.62362,
	0.62462,	0.62563,	0.62663,	0.62763,
0.62863,	0.62963,	0.63063,	0.63163,	0.63263,
0.63363,	0.63463,	0.63564,	0.63664,	0.63764,
0.63864,	0.63964,	0.64064,	0.64164,	0.64264,
0.64364,	0.64464,	0.64565,	0.64665,	0.64765,
	0.64865,	0.64965,	0.65065,	0.65165,
0.65265,	0.65365,	0.65465,	0.65566,	0.65666,
0.65766,	0.65866,	0.65966,	0.66066,	0.66166,
0.66266,	0.66366,	0.66466,	0.66567,	0.66667,
0.66767,	0.66867,	0.66967,	0.67067,	0.67167,
	0.67267,	0.67367,	0.67467,	0.67568,
0.67668,	0.67768,	0.67868,	0.67968,	0.68068,
0.68168,	0.68268,	0.68368,	0.68468,	0.68569,
0.68669,	0.68769,	0.68869,	0.68969,	0.69069,
0.69169,	0.69269,	0.69369,	0.69469,	0.6957,
	0.6967,	0.6977,	0.6987,	0.6997,
0.7007,	0.7017,	0.7027,	0.7037,	0.7047,
0.70571,	0.70671,	0.70771,	0.70871,	0.70971,
0.71071,	0.71171,	0.71271,	0.71371,	0.71471,
0.71572,	0.71672,	0.71772,	0.71872,	0.71972,
	0.72072,	0.72172,	0.72272,	0.72372,
0.72472,	0.72573,	0.72673,	0.72773,	0.72873,
0.72973,	0.73073,	0.73173,	0.73273,	0.73373,
0.73473,	0.73574,	0.73674,	0.73774,	0.73874,
0.73974,	0.74074,	0.74174,	0.74274,	0.74374,
	0.74474,	0.74575,	0.74675,	0.74775,
0.74875,	0.74975,	0.75075,	0.75175,	0.75275,
0.75375,	0.75475,	0.75576,	0.75676,	0.75776,
0.75876,	0.75976,	0.76076,	0.76176,	0.76276,
0.76376,	0.76476,	0.76577,	0.76677,	0.76777,
	0.76877,	0.76977,	0.77077,	0.77177,
0.77277,	0.77377,	0.77477,	0.77578,	0.77678,
0.77778,	0.77878,	0.77978,	0.78078,	0.78178,
0.78278,	0.78378,	0.78478,	0.78579,	0.78679,
0.78779,	0.78879,	0.78979,	0.79079,	0.79179,
	0.79279,	0.79379,	0.79479,	0.7958,
0.7968,	0.7978,	0.7988,	0.7998,	0.8008,
0.8018,	0.8028,	0.8038,	0.8048,	0.80581,
0.80681,	0.80781,	0.80881,	0.80981,	0.81081,
0.81181,	0.81281,	0.81381,	0.81481,	0.81582,
	0.81682,	0.81782,	0.81882,	0.81982,
0.82082,	0.82182,	0.82282,	0.82382,	0.82482,
0.82583,	0.82683,	0.82783,	0.82883,	0.82983,
0.83083,	0.83183,	0.83283,	0.83383,	0.83483,
0.83584,	0.83684,	0.83784,	0.83884,	0.83984,
	0.84084,	0.84184,	0.84284,	0.84384,
0.84484,	0.84585,	0.84685,	0.84785,	0.84885,
0.84985,	0.85085,	0.85185,	0.85285,	0.85385,

0.85485,	0.85586,	0.85686,	0.85786,	0.85886,	
0.85986,	0.86086,	0.86186,	0.86286,	0.86386,	
	0.86486,	0.86587,	0.86687,	0.86787,	
0.86887,	0.86987,	0.87087,	0.87187,	0.87287,	
0.87387,	0.87487,	0.87588,	0.87688,	0.87788,	
0.87888,	0.87988,	0.88088,	0.88188,	0.88288,	
0.88388,	0.88488,	0.88589,	0.88689,	0.88789,	
	0.88889,	0.88989,	0.89089,	0.89189,	
0.89289,	0.89389,	0.89489,	0.8959,	0.8969,	
0.8979,	0.8989,	0.8999,	0.9009,	0.9019,	
0.9029,	0.9039,	0.9049,	0.90591,	0.90691,	
0.90791,	0.90891,	0.90991,	0.91091,	0.91191,	
	0.91291,	0.91391,	0.91491,	0.91592,	
0.91692,	0.91792,	0.91892,	0.91992,	0.92092,	
0.92192,	0.92292,	0.92392,	0.92492,	0.92593,	
0.92693,	0.92793,	0.92893,	0.92993,	0.93093,	
0.93193,	0.93293,	0.93393,	0.93493,	0.93594,	
	0.93694,	0.93794,	0.93894,	0.93994,	
0.94094,	0.94194,	0.94294,	0.94394,	0.94494,	
0.94595,	0.94695,	0.94795,	0.94895,	0.94995,	
0.95095,	0.95195,	0.95295,	0.95395,	0.95495,	
0.95596,	0.95696,	0.95796,	0.95896,	0.95996,	
	0.96096,	0.96196,	0.96296,	0.96396,	
0.96496,	0.96597,	0.96697,	0.96797,	0.96897,	
0.96997,	0.97097,	0.97197,	0.97297,	0.97397,	
0.97497,	0.97598,	0.97698,	0.97798,	0.97898,	
0.97998,	0.98098,	0.98198,	0.98298,	0.98398,	
	0.98498,	0.98599,	0.98699,	0.98799,	
0.98899,	0.98999,	0.99099,	0.99199,	0.99299,	
0.99399,	0.99499,	0.996,	0.997,	0.998,	
0.999,	1]],	array([[	0.15066,	0.15076,	0.20978,
0.251,	0.28161,	0.30814,	0.33122,	0.35103,	
0.36822,	0.38445,	0.39943,	0.41376,	0.42799,	
0.44159,	0.45185,	0.4617,	0.47137,	0.48026,	
0.49,	0.49929,	0.50793,	0.51637,	0.52197,	
	0.5285,	0.5362,	0.54353,	0.54943,	
0.55493,	0.56006,	0.56529,	0.56894,	0.57407,	
0.57837,	0.58518,	0.58918,	0.59281,	0.59638,	
0.60197,	0.60635,	0.61003,	0.61354,	0.61779,	
0.6212,	0.62522,	0.62782,	0.63155,		
	0.6336,	0.63719,	0.64102,	0.64345,	
0.64642,	0.65,	0.65264,	0.65616,	0.65826,	
0.66068,	0.66263,	0.66584,	0.66686,	0.67066,	
0.67339,	0.67534,	0.67767,	0.67898,	0.68143,	
0.68217,	0.68465,	0.68736,	0.68912,		
	0.69186,	0.69338,	0.69683,	0.69833,	
0.701,	0.70322,	0.70441,	0.70663,	0.70888,	
0.71001,	0.71256,	0.71541,	0.71591,	0.71897,	
0.72079,	0.72202,	0.72393,	0.72501,	0.72608,	

0.72702,	0.72833,	0.72978,	0.73143,	
	0.7335,	0.7349,	0.7355,	0.73759,
0.7383,	0.73907,	0.73944,	0.7407,	0.74193,
0.74258,	0.74341,	0.74467,	0.74467,	0.74518,
0.74621,	0.74678,	0.74795,	0.74944,	0.75097,
0.75297,	0.75343,	0.75412,	0.75498,	
	0.75506,	0.75642,	0.75683,	0.75793,
0.75847,	0.75903,	0.76009,	0.76078,	0.76144,
0.76175,	0.76302,	0.76416,	0.76586,	0.76713,
0.76776,	0.76876,	0.77039,	0.77111,	0.77197,
0.77211,	0.77261,	0.77408,	0.77459,	
	0.77497,	0.77527,	0.77516,	0.77623,
0.77669,	0.77714,	0.77774,	0.77775,	0.77873,
0.77909,	0.77932,	0.78041,	0.78071,	0.78108,
0.78117,	0.78126,	0.7818,	0.78263,	0.78312,
0.78349,	0.78375,	0.78445,	0.78455,	
	0.78464,	0.78572,	0.78622,	0.78675,
0.78725,	0.78804,	0.78831,	0.78879,	0.78952,
0.7893,	0.78917,	0.78992,	0.79049,	0.79044,
0.79079,	0.79143,	0.79183,	0.79213,	0.79263,
0.79356,	0.79422,	0.79422,	0.79448,	
	0.79459,	0.79466,	0.79531,	0.79522,
0.79562,	0.79605,	0.79613,	0.79622,	0.7966,
0.79689,	0.79781,	0.79813,	0.79896,	0.79932,
0.79996,	0.80077,	0.80096,	0.80109,	0.80128,
0.80211,	0.8023,	0.8025,	0.80277,	
	0.80241,	0.8022,	0.80248,	0.8032,
0.80367,	0.8039,	0.80419,	0.80471,	0.8055,
0.80557,	0.80564,	0.80571,	0.80606,	0.80652,
0.80667,	0.80675,	0.80673,	0.807,	0.80773,
0.80855,	0.80909,	0.80945,	0.80999,	
	0.80998,	0.81024,	0.8104,	0.81052,
0.81044,	0.81057,	0.81035,	0.80993,	0.81029,
0.81018,	0.80971,	0.80965,	0.80964,	0.81004,
0.80994,	0.81038,	0.81012,	0.81085,	0.81081,
0.81089,	0.81188,	0.81233,	0.8126,	
	0.81259,	0.81273,	0.81309,	0.81353,
0.81385,	0.81399,	0.81418,	0.81435,	0.81406,
0.8142,	0.81448,	0.81424,	0.81432,	0.81446,
0.81469,	0.81518,	0.81536,	0.81549,	0.81566,
0.81609,	0.81595,	0.81627,	0.8169,	
	0.81725,	0.81721,	0.81703,	0.81664,
0.81744,	0.81779,	0.81722,	0.81803,	0.81809,
0.81827,	0.8184,	0.81841,	0.81868,	0.81843,
0.81785,	0.81721,	0.81729,	0.81792,	0.81729,
0.81745,	0.81807,	0.81825,	0.81776,	
	0.81808,	0.81816,	0.81824,	0.81832,
0.81811,	0.81796,	0.81827,	0.81808,	0.81827,
0.81818,	0.81804,	0.8179,	0.81789,	0.81772,

0.81796,	0.81847,	0.81865,	0.81874,	0.81884,
0.81918,	0.81866,	0.81875,	0.81885,	
	0.81867,	0.81931,	0.81924,	0.81962,
0.82023,	0.81988,	0.82017,	0.82064,	0.82058,
0.8205,	0.82125,	0.82147,	0.82208,	0.82277,
0.82247,	0.82326,	0.82324,	0.82372,	0.82393,
0.82501,	0.82577,	0.82604,	0.82641,	
	0.82661,	0.82639,	0.82676,	0.82703,
0.8274,	0.8278,	0.82794,	0.82775,	0.82853,
0.82864,	0.82876,	0.82862,	0.82841,	0.82807,
0.82802,	0.82817,	0.82792,	0.82801,	0.82722,
0.82712,	0.82625,	0.82644,	0.82662,	
	0.82637,	0.82679,	0.82709,	0.82722,
0.82739,	0.82724,	0.82719,	0.82713,	0.82665,
0.8269,	0.82657,	0.82672,	0.82667,	0.82607,
0.82618,	0.82624,	0.82556,	0.8254,	0.8259,
0.82614,	0.82622,	0.82587,	0.82566,	
	0.82561,	0.82583,	0.82618,	0.82603,
0.8256,	0.82542,	0.82515,	0.82485,	0.82502,
0.82517,	0.82554,	0.82635,	0.8262,	0.82574,
0.82596,	0.82637,	0.82681,	0.82724,	0.82748,
0.82782,	0.82804,	0.82823,	0.82799,	
	0.82819,	0.82781,	0.82781,	0.82756,
0.82731,	0.82744,	0.82721,	0.82736,	0.82772,
0.82728,	0.82709,	0.82706,	0.82633,	0.82583,
0.82581,	0.82545,	0.82564,	0.82536,	0.8248,
0.82419,	0.82388,	0.82309,	0.8231,	
	0.82296,	0.82282,	0.82167,	0.82219,
0.82206,	0.82237,	0.82259,	0.8228,	0.82296,
0.82313,	0.8233,	0.82242,	0.82226,	0.82255,
0.82192,	0.82203,	0.82216,	0.82127,	0.82095,
0.82076,	0.82061,	0.82047,	0.82042,	
	0.82023,	0.82046,	0.81926,	0.81897,
0.81903,	0.81884,	0.81898,	0.81837,	0.8178,
0.81781,	0.81801,	0.81781,	0.81764,	0.81853,
0.8184,	0.81692,	0.81608,	0.81628,	0.81581,
0.81635,	0.81627,	0.81585,	0.81544,	
	0.81532,	0.81502,	0.81426,	0.81294,
0.81254,	0.81232,	0.81185,	0.81196,	0.81206,
0.81216,	0.81118,	0.81133,	0.81106,	0.81118,
0.81101,	0.8108,	0.81048,	0.81012,	0.81039,
0.81024,	0.81009,	0.81026,	0.81,	
	0.80986,	0.8089,	0.80838,	0.80779,
0.80742,	0.80698,	0.80742,	0.80723,	0.80704,
0.80683,	0.80661,	0.8067,	0.80609,	0.80671,
0.80703,	0.80701,	0.80665,	0.8067,	0.80702,
0.80701,	0.80687,	0.80584,	0.80603,	
	0.80588,	0.80573,	0.80486,	0.80495,
0.80508,	0.8044,	0.804,	0.80418,	0.80487,

0.80473,	0.804,	0.80286,	0.80286,	0.80181,
0.80165,	0.80083,	0.80057,	0.80015,	0.79978,
0.79976,	0.79991,	0.79848,	0.79702,	
	0.79534,	0.79446,	0.79462,	0.79328,
0.79273,	0.7925,	0.79146,	0.79134,	0.79145,
0.79045,	0.78834,	0.78682,	0.78553,	0.78496,
0.78422,	0.78393,	0.78333,	0.78302,	0.78224,
0.78177,	0.78092,	0.78034,	0.78002,	
	0.77912,	0.7789,	0.77754,	0.77672,
0.77634,	0.77606,	0.77616,	0.77496,	0.77422,
0.77478,	0.7743,	0.77364,	0.7724,	0.77186,
0.77055,	0.76812,	0.76723,	0.76774,	0.7661,
0.76614,	0.76536,	0.76349,	0.76317,	
	0.76176,	0.76116,	0.75878,	0.7574,
0.75605,	0.7553,	0.75383,	0.75326,	0.75184,
0.74788,	0.74758,	0.74701,	0.74461,	0.74391,
0.7436,	0.74183,	0.7401,	0.73819,	0.73667,
0.7361,	0.73491,	0.73423,	0.73225,	
	0.72924,	0.72779,	0.72525,	0.72375,
0.7224,	0.72047,	0.71979,	0.7183,	0.71592,
0.71504,	0.71297,	0.71088,	0.70913,	0.7082,
0.70754,	0.70636,	0.70354,	0.70202,	0.6989,
0.69702,	0.69291,	0.69101,	0.68834,	
	0.68483,	0.68353,	0.68144,	0.67847,
0.67722,	0.67589,	0.6747,	0.67296,	0.6723,
0.67187,	0.67018,	0.66699,	0.66503,	0.66279,
0.66112,	0.65923,	0.65726,	0.65289,	0.65051,
0.64803,	0.64674,	0.64389,	0.64006,	
	0.63701,	0.63578,	0.63391,	0.63077,
0.62842,	0.62524,	0.62322,	0.62187,	0.62047,
0.61771,	0.61514,	0.61201,	0.6084,	0.60615,
0.60466,	0.60226,	0.60046,	0.59823,	0.59038,
0.5881,	0.58469,	0.58007,	0.57381,	
	0.56958,	0.56508,	0.56266,	0.55898,
0.55443,	0.54848,	0.54366,	0.53994,	0.53849,
0.53504,	0.52993,	0.526,	0.5211,	0.51738,
0.50833,	0.50264,	0.49333,	0.48276,	0.47559,
0.46929,	0.46387,	0.45389,	0.44803,	
	0.4423,	0.43516,	0.42918,	0.42346,
0.41567,	0.40706,	0.39486,	0.38735,	0.3815,
0.36964,	0.36242,	0.3558,	0.34856,	0.33863,
0.33354,	0.33006,	0.32353,	0.3136,	0.30709,
0.30082,	0.29219,	0.28018,	0.27606,	
	0.27024,	0.26224,	0.25381,	0.24996,
0.24161,	0.23553,	0.23243,	0.22813,	0.21546,
0.20789,	0.20441,	0.20387,	0.2001,	0.19565,
0.1902,	0.18396,	0.17827,	0.1752,	0.1708,
0.16751,	0.15942,	0.15403,	0.14949,	
	0.14213,	0.1415,	0.13777,	0.13612,

0.13104,	0.12719,	0.12641,	0.12381,	0.12303,	
0.11996,	0.11746,	0.11308,	0.10903,	0.10787,	
0.10476,	0.10124,	0.098457,	0.09419,	0.091007,	
0.087413,	0.084875,	0.083597,	0.082995,		
	0.078289,	0.074841,	0.072514,	0.069735,	
0.067056,	0.066261,	0.065932,	0.065604,	0.065275,	
0.062756,	0.062079,	0.06026,	0.059268,	0.057992,	
0.057543,	0.057094,	0.056096,	0.053627,	0.052484,	
0.051582,	0.050578,	0.046881,	0.045573,		
	0.041975,	0.039617,	0.038991,	0.037402,	
0.037193,	0.036983,	0.036774,	0.036564,	0.036354,	
0.034867,	0.034084,	0.032743,	0.030735,	0.029922,	
0.028649,	0.02748,	0.02712,	0.026759,	0.026398,	
0.024534,	0.023738,	0.022428,	0.021956,		
	0.021483,	0.020754,	0.018889,	0.018282,	
0.017674,	0.014654,	0.013766,	0.013096,	0.011905,	
0.009944,	0.0095897,	0.0092354,	0.0088809,	0.0069574,	
0.0052069,	0.0038452,	0.0032249,	0.0026249,	0.0023099,	
0.0020421,	0.0017743,	0.0015063,	0,		
	0,	0,	0,	0,	
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
	0,	0,	0,	0,	
	0,	0,	0,	0,	
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
	0,	0,	0,	0,	
	0,	0,	0,	0,	
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
	0,	0,	0,	0,	
	0,	0,	0,	0,	
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
	0,	0,	0,	0,	
	0,	0,	0,	0,	
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
	0,	0,	0,	0,	
	0,	0,	0,	0,	
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
	0,	0,	0,	0,	
	0,	0,	0,	0,	
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
	0,	0,	0,	0,	
	0,	0,	0,	0,	
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
	0,	0,	0,	0,	
	0,	0,	0,	0,	
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
	0,	0,	0,	0,	
	0,	0,	0,	0,	
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
	0,	0,	0,	0,	
	0,	0,	0,	0,	
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
	0,	0,	0,	0,	
	0,	0,	0,	0,	
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
	0,	0,	0,	0,	
	0,	0,	0,	0,	
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
	0,	0,	0,	0,	
	0,	0,	0,	0,	
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
	0,	0,	0,	0,	
	0,	0,	0,	0,	
0,	0,	0,	0,	0,	0,
0,	0,	0			

0,	0,	0,	0,	0,	0,	0,
0]], 'Confidence', 'F1'], [array([	0,	0.001001,				
0.002002,	0.003003,	0.004004,	0.005005,	0.006006,		
0.007007,	0.008008,	0.009009,	0.01001,	0.011011,		
0.012012,	0.013013,	0.014014,	0.015015,	0.016016,		
0.017017,	0.018018,	0.019019,	0.02002,	0.021021,		
0.022022,	0.023023,					
	0.024024,	0.025025,	0.026026,	0.027027,		
0.028028,	0.029029,	0.03003,	0.031031,	0.032032,		
0.033033,	0.034034,	0.035035,	0.036036,	0.037037,		
0.038038,	0.039039,	0.04004,	0.041041,	0.042042,		
0.043043,	0.044044,	0.045045,	0.046046,	0.047047,		
	0.048048,	0.049049,	0.05005,	0.051051,		
0.052052,	0.053053,	0.054054,	0.055055,	0.056056,		
0.057057,	0.058058,	0.059059,	0.06006,	0.061061,		
0.062062,	0.063063,	0.064064,	0.065065,	0.066066,		
0.067067,	0.068068,	0.069069,	0.07007,	0.071071,		
	0.072072,	0.073073,	0.074074,	0.075075,		
0.076076,	0.077077,	0.078078,	0.079079,	0.08008,		
0.081081,	0.082082,	0.083083,	0.084084,	0.085085,		
0.086086,	0.087087,	0.088088,	0.089089,	0.09009,		
0.091091,	0.092092,	0.093093,	0.094094,	0.095095,		
	0.096096,	0.097097,	0.098098,	0.099099,		
0.1001,	0.1011,	0.1021,	0.1031,	0.1041,		
0.10511,	0.10611,	0.10711,	0.10811,	0.10911,		
0.11011,	0.11111,	0.11211,	0.11311,	0.11411,		
0.11512,	0.11612,	0.11712,	0.11812,	0.11912,		
	0.12012,	0.12112,	0.12212,	0.12312,		
0.12412,	0.12513,	0.12613,	0.12713,	0.12813,		
0.12913,	0.13013,	0.13113,	0.13213,	0.13313,		
0.13413,	0.13514,	0.13614,	0.13714,	0.13814,		
0.13914,	0.14014,	0.14114,	0.14214,	0.14314,		
	0.14414,	0.14515,	0.14615,	0.14715,		
0.14815,	0.14915,	0.15015,	0.15115,	0.15215,		
0.15315,	0.15415,	0.15516,	0.15616,	0.15716,		
0.15816,	0.15916,	0.16016,	0.16116,	0.16216,		
0.16316,	0.16416,	0.16517,	0.16617,	0.16717,		
	0.16817,	0.16917,	0.17017,	0.17117,		
0.17217,	0.17317,	0.17417,	0.17518,	0.17618,		
0.17718,	0.17818,	0.17918,	0.18018,	0.18118,		
0.18218,	0.18318,	0.18418,	0.18519,	0.18619,		
0.18719,	0.18819,	0.18919,	0.19019,	0.19119,		
	0.19219,	0.19319,	0.19419,	0.1952,		
0.1962,	0.1972,	0.1982,	0.1992,	0.2002,		
0.2012,	0.2022,	0.2032,	0.2042,	0.20521,		
0.20621,	0.20721,	0.20821,	0.20921,	0.21021,		
0.21121,	0.21221,	0.21321,	0.21421,	0.21522,		
	0.21622,	0.21722,	0.21822,	0.21922,		

0.22022,	0.22122,	0.22222,	0.22322,	0.22422,
0.22523,	0.22623,	0.22723,	0.22823,	0.22923,
0.23023,	0.23123,	0.23223,	0.23323,	0.23423,
0.23524,	0.23624,	0.23724,	0.23824,	0.23924,
	0.24024,	0.24124,	0.24224,	0.24324,
0.24424,	0.24525,	0.24625,	0.24725,	0.24825,
0.24925,	0.25025,	0.25125,	0.25225,	0.25325,
0.25425,	0.25526,	0.25626,	0.25726,	0.25826,
0.25926,	0.26026,	0.26126,	0.26226,	0.26326,
	0.26426,	0.26527,	0.26627,	0.26727,
0.26827,	0.26927,	0.27027,	0.27127,	0.27227,
0.27327,	0.27427,	0.27528,	0.27628,	0.27728,
0.27828,	0.27928,	0.28028,	0.28128,	0.28228,
0.28328,	0.28428,	0.28529,	0.28629,	0.28729,
	0.28829,	0.28929,	0.29029,	0.29129,
0.29229,	0.29329,	0.29429,	0.2953,	0.2963,
0.2973,	0.2983,	0.2993,	0.3003,	0.3013,
0.3023,	0.3033,	0.3043,	0.30531,	0.30631,
0.30731,	0.30831,	0.30931,	0.31031,	0.31131,
	0.31231,	0.31331,	0.31431,	0.31532,
0.31632,	0.31732,	0.31832,	0.31932,	0.32032,
0.32132,	0.32232,	0.32332,	0.32432,	0.32533,
0.32633,	0.32733,	0.32833,	0.32933,	0.33033,
0.33133,	0.33233,	0.33333,	0.33433,	0.33534,
	0.33634,	0.33734,	0.33834,	0.33934,
0.34034,	0.34134,	0.34234,	0.34334,	0.34434,
0.34535,	0.34635,	0.34735,	0.34835,	0.34935,
0.35035,	0.35135,	0.35235,	0.35335,	0.35435,
0.35536,	0.35636,	0.35736,	0.35836,	0.35936,
	0.36036,	0.36136,	0.36236,	0.36336,
0.36436,	0.36537,	0.36637,	0.36737,	0.36837,
0.36937,	0.37037,	0.37137,	0.37237,	0.37337,
0.37437,	0.37538,	0.37638,	0.37738,	0.37838,
0.37938,	0.38038,	0.38138,	0.38238,	0.38338,
	0.38438,	0.38539,	0.38639,	0.38739,
0.38839,	0.38939,	0.39039,	0.39139,	0.39239,
0.39339,	0.39439,	0.3954,	0.3964,	0.3974,
0.3984,	0.3994,	0.4004,	0.4014,	0.4024,
0.4034,	0.4044,	0.40541,	0.40641,	0.40741,
	0.40841,	0.40941,	0.41041,	0.41141,
0.41241,	0.41341,	0.41441,	0.41542,	0.41642,
0.41742,	0.41842,	0.41942,	0.42042,	0.42142,
0.42242,	0.42342,	0.42442,	0.42543,	0.42643,
0.42743,	0.42843,	0.42943,	0.43043,	0.43143,
	0.43243,	0.43343,	0.43443,	0.43544,
0.43644,	0.43744,	0.43844,	0.43944,	0.44044,
0.44144,	0.44244,	0.44344,	0.44444,	0.44545,
0.44645,	0.44745,	0.44845,	0.44945,	0.45045,
0.45145,	0.45245,	0.45345,	0.45445,	0.45546,

	0.45646,	0.45746,	0.45846,	0.45946,
0.46046,	0.46146,	0.46246,	0.46346,	0.46446,
0.46547,	0.46647,	0.46747,	0.46847,	0.46947,
0.47047,	0.47147,	0.47247,	0.47347,	0.47447,
0.47548,	0.47648,	0.47748,	0.47848,	0.47948,
	0.48048,	0.48148,	0.48248,	0.48348,
0.48448,	0.48549,	0.48649,	0.48749,	0.48849,
0.48949,	0.49049,	0.49149,	0.49249,	0.49349,
0.49449,	0.4955,	0.4965,	0.4975,	0.4985,
0.4995,	0.5005,	0.5015,	0.5025,	0.5035,
	0.5045,	0.50551,	0.50651,	0.50751,
0.50851,	0.50951,	0.51051,	0.51151,	0.51251,
0.51351,	0.51451,	0.51552,	0.51652,	0.51752,
0.51852,	0.51952,	0.52052,	0.52152,	0.52252,
0.52352,	0.52452,	0.52553,	0.52653,	0.52753,
	0.52853,	0.52953,	0.53053,	0.53153,
0.53253,	0.53353,	0.53453,	0.53554,	0.53654,
0.53754,	0.53854,	0.53954,	0.54054,	0.54154,
0.54254,	0.54354,	0.54454,	0.54555,	0.54655,
0.54755,	0.54855,	0.54955,	0.55055,	0.55155,
	0.55255,	0.55355,	0.55455,	0.55556,
0.55656,	0.55756,	0.55856,	0.55956,	0.56056,
0.56156,	0.56256,	0.56356,	0.56456,	0.56557,
0.56657,	0.56757,	0.56857,	0.56957,	0.57057,
0.57157,	0.57257,	0.57357,	0.57457,	0.57558,
	0.57658,	0.57758,	0.57858,	0.57958,
0.58058,	0.58158,	0.58258,	0.58358,	0.58458,
0.58559,	0.58659,	0.58759,	0.58859,	0.58959,
0.59059,	0.59159,	0.59259,	0.59359,	0.59459,
0.5956,	0.5966,	0.5976,	0.5986,	0.5996,
	0.6006,	0.6016,	0.6026,	0.6036,
0.6046,	0.60561,	0.60661,	0.60761,	0.60861,
0.60961,	0.61061,	0.61161,	0.61261,	0.61361,
0.61461,	0.61562,	0.61662,	0.61762,	0.61862,
0.61962,	0.62062,	0.62162,	0.62262,	0.62362,
	0.62462,	0.62563,	0.62663,	0.62763,
0.62863,	0.62963,	0.63063,	0.63163,	0.63263,
0.63363,	0.63463,	0.63564,	0.63664,	0.63764,
0.63864,	0.63964,	0.64064,	0.64164,	0.64264,
0.64364,	0.64464,	0.64565,	0.64665,	0.64765,
	0.64865,	0.64965,	0.65065,	0.65165,
0.65265,	0.65365,	0.65465,	0.65566,	0.65666,
0.65766,	0.65866,	0.65966,	0.66066,	0.66166,
0.66266,	0.66366,	0.66466,	0.66567,	0.66667,
0.66767,	0.66867,	0.66967,	0.67067,	0.67167,
	0.67267,	0.67367,	0.67467,	0.67568,
0.67668,	0.67768,	0.67868,	0.67968,	0.68068,
0.68168,	0.68268,	0.68368,	0.68468,	0.68569,
0.68669,	0.68769,	0.68869,	0.68969,	0.69069,

0.69169,	0.69269,	0.69369,	0.69469,	0.6957,
	0.6967,	0.6977,	0.6987,	0.6997,
0.7007,	0.7017,	0.7027,	0.7037,	0.7047,
0.70571,	0.70671,	0.70771,	0.70871,	0.70971,
0.71071,	0.71171,	0.71271,	0.71371,	0.71471,
0.71572,	0.71672,	0.71772,	0.71872,	0.71972,
	0.72072,	0.72172,	0.72272,	0.72372,
0.72472,	0.72573,	0.72673,	0.72773,	0.72873,
0.72973,	0.73073,	0.73173,	0.73273,	0.73373,
0.73473,	0.73574,	0.73674,	0.73774,	0.73874,
0.73974,	0.74074,	0.74174,	0.74274,	0.74374,
	0.74474,	0.74575,	0.74675,	0.74775,
0.74875,	0.74975,	0.75075,	0.75175,	0.75275,
0.75375,	0.75475,	0.75576,	0.75676,	0.75776,
0.75876,	0.75976,	0.76076,	0.76176,	0.76276,
0.76376,	0.76476,	0.76577,	0.76677,	0.76777,
	0.76877,	0.76977,	0.77077,	0.77177,
0.77277,	0.77377,	0.77477,	0.77578,	0.77678,
0.77778,	0.77878,	0.77978,	0.78078,	0.78178,
0.78278,	0.78378,	0.78478,	0.78579,	0.78679,
0.78779,	0.78879,	0.78979,	0.79079,	0.79179,
	0.79279,	0.79379,	0.79479,	0.7958,
0.7968,	0.7978,	0.7988,	0.7998,	0.8008,
0.8018,	0.8028,	0.8038,	0.8048,	0.80581,
0.80681,	0.80781,	0.80881,	0.80981,	0.81081,
0.81181,	0.81281,	0.81381,	0.81481,	0.81582,
	0.81682,	0.81782,	0.81882,	0.81982,
0.82082,	0.82182,	0.82282,	0.82382,	0.82482,
0.82583,	0.82683,	0.82783,	0.82883,	0.82983,
0.83083,	0.83183,	0.83283,	0.83383,	0.83483,
0.83584,	0.83684,	0.83784,	0.83884,	0.83984,
	0.84084,	0.84184,	0.84284,	0.84384,
0.84484,	0.84585,	0.84685,	0.84785,	0.84885,
0.84985,	0.85085,	0.85185,	0.85285,	0.85385,
0.85485,	0.85586,	0.85686,	0.85786,	0.85886,
0.85986,	0.86086,	0.86186,	0.86286,	0.86386,
	0.86486,	0.86587,	0.86687,	0.86787,
0.86887,	0.86987,	0.87087,	0.87187,	0.87287,
0.87387,	0.87487,	0.87588,	0.87688,	0.87788,
0.87888,	0.87988,	0.88088,	0.88188,	0.88288,
0.88388,	0.88488,	0.88589,	0.88689,	0.88789,
	0.88889,	0.88989,	0.89089,	0.89189,
0.89289,	0.89389,	0.89489,	0.8959,	0.8969,
0.8979,	0.8989,	0.8999,	0.9009,	0.9019,
0.9029,	0.9039,	0.9049,	0.90591,	0.90691,
0.90791,	0.90891,	0.90991,	0.91091,	0.91191,
	0.91291,	0.91391,	0.91491,	0.91592,
0.91692,	0.91792,	0.91892,	0.91992,	0.92092,
0.92192,	0.92292,	0.92392,	0.92492,	0.92593,
0.92693,	0.92793,	0.92893,	0.92993,	0.93093,

0.93193,	0.93293,	0.93393,	0.93493,	0.93594,
	0.93694,	0.93794,	0.93894,	0.93994,
0.94094,	0.94194,	0.94294,	0.94394,	0.94494,
0.94595,	0.94695,	0.94795,	0.94895,	0.94995,
0.95095,	0.95195,	0.95295,	0.95395,	0.95495,
0.95596,	0.95696,	0.95796,	0.95896,	0.95996,
	0.96096,	0.96196,	0.96296,	0.96396,
0.96496,	0.96597,	0.96697,	0.96797,	0.96897,
0.96997,	0.97097,	0.97197,	0.97297,	0.97397,
0.97497,	0.97598,	0.97698,	0.97798,	0.97898,
0.97998,	0.98098,	0.98198,	0.98298,	0.98398,
	0.98498,	0.98599,	0.98699,	0.98799,
0.98899,	0.98999,	0.99099,	0.99199,	0.99299,
0.99399,	0.99499,	0.996,	0.997,	0.998,
0.999,	1]],	array([[	0.08186,	0.081915,
				0.11802,
0.14483,	0.16576,	0.18451,	0.20131,	0.21617,
0.22936,	0.24222,	0.25428,	0.26601,	0.27789,
0.28946,	0.29848,	0.3072,	0.3159,	0.32394,
0.33286,	0.34159,	0.34972,	0.35787,	0.36357,
	0.37014,	0.37773,	0.38516,	0.39112,
0.39683,	0.4021,	0.40751,	0.41132,	0.41696,
0.42191,	0.42934,	0.4338,	0.43803,	0.44194,
0.44812,	0.4533,	0.45742,	0.46138,	0.46637,
0.47044,	0.47506,	0.47841,	0.48277,	
	0.48517,	0.48957,	0.4941,	0.49719,
0.50074,	0.50544,	0.50884,	0.51313,	0.51571,
0.51889,	0.5213,	0.52549,	0.5272,	0.53197,
0.53563,	0.53832,	0.54129,	0.54297,	0.54611,
0.54729,	0.55048,	0.55448,	0.55725,	
	0.56085,	0.5631,	0.56817,	0.57016,
0.57398,	0.57728,	0.5791,	0.5821,	0.58543,
0.58753,	0.59102,	0.59524,	0.59707,	0.60132,
0.60417,	0.6059,	0.6086,	0.61013,	0.61164,
0.61298,	0.61504,	0.61752,	0.61988,	
	0.62287,	0.62488,	0.62638,	0.62942,
0.6311,	0.63238,	0.6334,	0.63558,	0.63772,
0.63869,	0.64011,	0.64212,	0.64245,	0.64389,
0.64543,	0.64627,	0.64803,	0.65062,	0.65293,
0.65596,	0.65665,	0.6577,	0.65906,	
	0.65949,	0.66156,	0.66255,	0.66425,
0.66543,	0.66629,	0.66793,	0.66899,	0.67004,
0.67086,	0.67284,	0.67462,	0.67764,	0.67963,
0.681,	0.68297,	0.68554,	0.68707,	0.68844,
0.68904,	0.68985,	0.69219,	0.69341,	
	0.69441,	0.6953,	0.69598,	0.69845,
0.69921,	0.69993,	0.70091,	0.70133,	0.70294,
0.70392,	0.70473,	0.7065,	0.70742,	0.70802,
0.70817,	0.70832,	0.70921,	0.71058,	0.71139,
0.71199,	0.71242,	0.71358,	0.71374,	

	0.7139,	0.71569,	0.71652,	0.7174,
0.71823,	0.71955,	0.72,	0.7208,	0.72202,
0.72263,	0.72318,	0.72461,	0.72584,	0.72621,
0.72724,	0.72833,	0.72901,	0.72951,	0.73037,
0.73195,	0.73353,	0.73399,	0.73442,	
	0.73508,	0.73613,	0.73723,	0.73755,
0.73829,	0.73943,	0.73958,	0.73973,	0.74038,
0.7409,	0.74295,	0.74351,	0.74496,	0.74557,
0.7467,	0.7481,	0.74891,	0.74914,	0.74948,
0.75093,	0.75127,	0.75161,	0.75219,	
	0.75205,	0.75205,	0.75255,	0.75383,
0.75465,	0.75505,	0.75557,	0.75648,	0.75788,
0.758,	0.75813,	0.75825,	0.75888,	0.75969,
0.76031,	0.76068,	0.76105,	0.76155,	0.76284,
0.7648,	0.76578,	0.76643,	0.76739,	
	0.76788,	0.76835,	0.76865,	0.76894,
0.76974,	0.77001,	0.76993,	0.76984,	0.77061,
0.77081,	0.77075,	0.77089,	0.77139,	0.77229,
0.77299,	0.77384,	0.77385,	0.77517,	0.77544,
0.77579,	0.77758,	0.77843,	0.77891,	
	0.77943,	0.77968,	0.78036,	0.78116,
0.78175,	0.78201,	0.7829,	0.7833,	0.7832,
0.78348,	0.78401,	0.78411,	0.7848,	0.78506,
0.78549,	0.78639,	0.78673,	0.78698,	0.7873,
0.7881,	0.78876,	0.78953,	0.79071,	
	0.79138,	0.7915,	0.79144,	0.7913,
0.79284,	0.7935,	0.79355,	0.79528,	0.79577,
0.7961,	0.79658,	0.79694,	0.79747,	0.79738,
0.79758,	0.79737,	0.7982,	0.7996,	0.79941,
0.79973,	0.80092,	0.80126,	0.80148,	
	0.8021,	0.80225,	0.80241,	0.80257,
0.8029,	0.80304,	0.80377,	0.80412,	0.80482,
0.80539,	0.80534,	0.8053,	0.80588,	0.80607,
0.80722,	0.80822,	0.80856,	0.80875,	0.80894,
0.8096,	0.80979,	0.80998,	0.81017,	
	0.81042,	0.81168,	0.81202,	0.81291,
0.8141,	0.81436,	0.81523,	0.81616,	0.8163,
0.8165,	0.81799,	0.81842,	0.81964,	0.82134,
0.82168,	0.82326,	0.82385,	0.82482,	0.82522,
0.82782,	0.82957,	0.83013,	0.83087,	
	0.83128,	0.83142,	0.83223,	0.83278,
0.83353,	0.83434,	0.83462,	0.8355,	0.83714,
0.83737,	0.83761,	0.83762,	0.83756,	0.83793,
0.83811,	0.8384,	0.83836,	0.83875,	0.83959,
0.84004,	0.83984,	0.84023,	0.84128,	
	0.84127,	0.84231,	0.84294,	0.84322,
0.84357,	0.84394,	0.84452,	0.8451,	0.84522,
0.84676,	0.84671,	0.84702,	0.8476,	0.84753,
0.84798,	0.84906,	0.84888,	0.84917,	0.85024,

0.85074,	0.85106,	0.85097,	0.85091,	
	0.85105,	0.85151,	0.85307,	0.85354,
0.85343,	0.85338,	0.85331,	0.85377,	0.8543,
0.85517,	0.85671,	0.85846,	0.85873,	0.85914,
0.85982,	0.86146,	0.86241,	0.86335,	0.86388,
0.86461,	0.86511,	0.86551,	0.86575,	
	0.8662,	0.86652,	0.86706,	0.86785,
0.86802,	0.86836,	0.86861,	0.86895,	0.86975,
0.86964,	0.8696,	0.86983,	0.86997,	0.86985,
0.87094,	0.87093,	0.87135,	0.87139,	0.87126,
0.87112,	0.87105,	0.87116,	0.87142,	
	0.87139,	0.87136,	0.87109,	0.87314,
0.87366,	0.87435,	0.87485,	0.87533,	0.87569,
0.87607,	0.87647,	0.87632,	0.87652,	0.87749,
0.87735,	0.87764,	0.87795,	0.87778,	0.87771,
0.87766,	0.87768,	0.8782,	0.87891,	
	0.88013,	0.88107,	0.88139,	0.88142,
0.88193,	0.88197,	0.88251,	0.88237,	0.88225,
0.88297,	0.88348,	0.88344,	0.88344,	0.88552,
0.88607,	0.88623,	0.88605,	0.88716,	0.8887,
0.88997,	0.89034,	0.89087,	0.89078,	
	0.89108,	0.89132,	0.89178,	0.89152,
0.89162,	0.89264,	0.89269,	0.89388,	0.89411,
0.89434,	0.89473,	0.89558,	0.89617,	0.89683,
0.89679,	0.89675,	0.89669,	0.8968,	0.89747,
0.89792,	0.89862,	0.89921,	0.89933,	
	0.89978,	0.8996,	0.90008,	0.90055,
0.90128,	0.90119,	0.90259,	0.90255,	0.90252,
0.90248,	0.90245,	0.90311,	0.90309,	0.90465,
0.90546,	0.90582,	0.90576,	0.90658,	0.90739,
0.90783,	0.90848,	0.90838,	0.90901,	
	0.90898,	0.90896,	0.90881,	0.90913,
0.90947,	0.90941,	0.91077,	0.91224,	0.91403,
0.91427,	0.91415,	0.91396,	0.91466,	0.91449,
0.915,	0.91503,	0.91499,	0.91492,	0.91556,
0.91637,	0.91678,	0.91677,	0.91725,	
	0.91698,	0.9174,	0.91903,	0.91954,
0.92048,	0.92161,	0.92146,	0.92217,	0.92293,
0.92328,	0.92321,	0.92298,	0.92279,	0.9227,
0.92259,	0.92255,	0.92321,	0.92317,	0.92305,
0.92298,	0.92285,	0.92317,	0.92424,	
	0.92572,	0.92638,	0.92618,	0.92684,
0.92678,	0.92674,	0.92828,	0.92893,	0.92883,
0.93127,	0.932,	0.93351,	0.93335,	0.93328,
0.93311,	0.93279,	0.934,	0.93553,	0.93592,
0.93661,	0.93652,	0.93628,	0.93781,	
	0.9394,	0.93933,	0.93904,	0.93944,
0.93956,	0.94033,	0.94107,	0.94181,	0.94164,
0.94117,	0.94114,	0.94107,	0.94104,	0.94178,

0.94332,	0.94312,	0.94292,	0.9436,	0.94433,	
0.94426,	0.94413,	0.94496,	0.94474,		
	0.9444,	0.94609,	0.94675,	0.94659,	
0.94738,	0.94717,	0.9471,	0.94694,	0.94764,	
0.94754,	0.94748,	0.94806,	0.94787,	0.94819,	
0.94868,	0.94955,	0.94925,	0.94909,	0.94876,	
0.94856,	0.94811,	0.9479,	0.94761,		
	0.94722,	0.94708,	0.94789,	0.94756,	
0.94848,	0.94833,	0.9482,	0.94801,	0.94794,	
0.94896,	0.95093,	0.95059,	0.95039,	0.95015,	
0.94997,	0.94977,	0.95178,	0.95132,	0.95107,	
0.95195,	0.95182,	0.95152,	0.95111,		
	0.95078,	0.95065,	0.95163,	0.95168,	
0.95224,	0.9519,	0.9529,	0.95276,	0.95384,	
0.95355,	0.95453,	0.95547,	0.95511,	0.9561,	
0.95601,	0.95577,	0.95559,	0.95799,	0.95844,	
0.95835,	0.95802,	0.95757,	0.95884,		
	0.95934,	0.95891,	0.95867,	0.95831,	
0.96081,	0.96024,	0.9613,	0.96095,	0.96236,	
0.96361,	0.96315,	0.9644,	0.96559,	0.96527,	
0.96447,	0.96395,	0.96307,	0.96204,	0.96316,	
0.96253,	0.96198,	0.96093,	0.96429,		
	0.96575,	0.96506,	0.96446,	0.96387,	
0.96304,	0.96209,	0.96068,	0.96216,	0.96392,	
0.96255,	0.96167,	0.96605,	0.96537,	0.96705,	
0.96647,	0.97202,	0.97445,	0.97673,	0.97948,	
0.97899,	0.97827,	0.97721,	0.97682,		
	0.97626,	0.97544,	0.97452,	0.97409,	
0.97309,	0.97232,	0.97191,	0.97133,	0.96949,	
0.96828,	0.9677,	0.96761,	0.96695,	0.96614,	
0.96511,	0.96384,	0.96262,	0.96193,	0.9609,	
0.9665,	0.96471,	0.9705,	0.96956,		
	0.96791,	0.97454,	0.97491,	0.9746,	
0.98221,	0.98164,	0.98153,	0.98112,	0.98099,	
0.98049,	0.98005,	0.97925,	0.9891,	0.98898,	
0.9896,	1,	1,	1,	1,	
1,	1,	1,	1,		
	1,	1,	1,	1,	
1,	1,	1,	1,	1,	1,
1,	1,	1,	1,	1,	1,
1,	1,	1,	1,	1,	1,
1,					
	1,	1,	1,	1,	
1,	1,	1,	1,	1,	1,
1,	1,	1,	1,	1,	1,
1,	1,	1,	1,	1,	1,
1,					
	1,	1,	1,	1,	
1,	1,	1,	1,	1,	1,

[illegible]

0.062062,	0.063063,	0.064064,	0.065065,	0.066066,
0.067067,	0.068068,	0.069069,	0.07007,	0.071071,
	0.072072,	0.073073,	0.074074,	0.075075,
0.076076,	0.077077,	0.078078,	0.079079,	0.08008,
0.081081,	0.082082,	0.083083,	0.084084,	0.085085,
0.086086,	0.087087,	0.088088,	0.089089,	0.09009,
0.091091,	0.092092,	0.093093,	0.094094,	0.095095,
	0.096096,	0.097097,	0.098098,	0.099099,
0.1001,	0.1011,	0.1021,	0.1031,	0.1041,
0.10511,	0.10611,	0.10711,	0.10811,	0.10911,
0.11011,	0.11111,	0.11211,	0.11311,	0.11411,
0.11512,	0.11612,	0.11712,	0.11812,	0.11912,
	0.12012,	0.12112,	0.12212,	0.12312,
0.12412,	0.12513,	0.12613,	0.12713,	0.12813,
0.12913,	0.13013,	0.13113,	0.13213,	0.13313,
0.13413,	0.13514,	0.13614,	0.13714,	0.13814,
0.13914,	0.14014,	0.14114,	0.14214,	0.14314,
	0.14414,	0.14515,	0.14615,	0.14715,
0.14815,	0.14915,	0.15015,	0.15115,	0.15215,
0.15315,	0.15415,	0.15516,	0.15616,	0.15716,
0.15816,	0.15916,	0.16016,	0.16116,	0.16216,
0.16316,	0.16416,	0.16517,	0.16617,	0.16717,
	0.16817,	0.16917,	0.17017,	0.17117,
0.17217,	0.17317,	0.17417,	0.17518,	0.17618,
0.17718,	0.17818,	0.17918,	0.18018,	0.18118,
0.18218,	0.18318,	0.18418,	0.18519,	0.18619,
0.18719,	0.18819,	0.18919,	0.19019,	0.19119,
	0.19219,	0.19319,	0.19419,	0.1952,
0.1962,	0.1972,	0.1982,	0.1992,	0.2002,
0.2012,	0.2022,	0.2032,	0.2042,	0.20521,
0.20621,	0.20721,	0.20821,	0.20921,	0.21021,
0.21121,	0.21221,	0.21321,	0.21421,	0.21522,
	0.21622,	0.21722,	0.21822,	0.21922,
0.22022,	0.22122,	0.22222,	0.22322,	0.22422,
0.22523,	0.22623,	0.22723,	0.22823,	0.22923,
0.23023,	0.23123,	0.23223,	0.23323,	0.23423,
0.23524,	0.23624,	0.23724,	0.23824,	0.23924,
	0.24024,	0.24124,	0.24224,	0.24324,
0.24424,	0.24525,	0.24625,	0.24725,	0.24825,
0.24925,	0.25025,	0.25125,	0.25225,	0.25325,
0.25425,	0.25526,	0.25626,	0.25726,	0.25826,
0.25926,	0.26026,	0.26126,	0.26226,	0.26326,
	0.26426,	0.26527,	0.26627,	0.26727,
0.26827,	0.26927,	0.27027,	0.27127,	0.27227,
0.27327,	0.27427,	0.27528,	0.27628,	0.27728,
0.27828,	0.27928,	0.28028,	0.28128,	0.28228,
0.28328,	0.28428,	0.28529,	0.28629,	0.28729,
	0.28829,	0.28929,	0.29029,	0.29129,
0.29229,	0.29329,	0.29429,	0.2953,	0.2963,

0.2973,	0.2983,	0.2993,	0.3003,	0.3013,
0.3023,	0.3033,	0.3043,	0.30531,	0.30631,
0.30731,	0.30831,	0.30931,	0.31031,	0.31131,
	0.31231,	0.31331,	0.31431,	0.31532,
0.31632,	0.31732,	0.31832,	0.31932,	0.32032,
0.32132,	0.32232,	0.32332,	0.32432,	0.32533,
0.32633,	0.32733,	0.32833,	0.32933,	0.33033,
0.33133,	0.33233,	0.33333,	0.33433,	0.33534,
	0.33634,	0.33734,	0.33834,	0.33934,
0.34034,	0.34134,	0.34234,	0.34334,	0.34434,
0.34535,	0.34635,	0.34735,	0.34835,	0.34935,
0.35035,	0.35135,	0.35235,	0.35335,	0.35435,
0.35536,	0.35636,	0.35736,	0.35836,	0.35936,
	0.36036,	0.36136,	0.36236,	0.36336,
0.36436,	0.36537,	0.36637,	0.36737,	0.36837,
0.36937,	0.37037,	0.37137,	0.37237,	0.37337,
0.37437,	0.37538,	0.37638,	0.37738,	0.37838,
0.37938,	0.38038,	0.38138,	0.38238,	0.38338,
	0.38438,	0.38539,	0.38639,	0.38739,
0.38839,	0.38939,	0.39039,	0.39139,	0.39239,
0.39339,	0.39439,	0.3954,	0.3964,	0.3974,
0.3984,	0.3994,	0.4004,	0.4014,	0.4024,
0.4034,	0.4044,	0.40541,	0.40641,	0.40741,
	0.40841,	0.40941,	0.41041,	0.41141,
0.41241,	0.41341,	0.41441,	0.41542,	0.41642,
0.41742,	0.41842,	0.41942,	0.42042,	0.42142,
0.42242,	0.42342,	0.42442,	0.42543,	0.42643,
0.42743,	0.42843,	0.42943,	0.43043,	0.43143,
	0.43243,	0.43343,	0.43443,	0.43544,
0.43644,	0.43744,	0.43844,	0.43944,	0.44044,
0.44144,	0.44244,	0.44344,	0.44444,	0.44545,
0.44645,	0.44745,	0.44845,	0.44945,	0.45045,
0.45145,	0.45245,	0.45345,	0.45445,	0.45546,
	0.45646,	0.45746,	0.45846,	0.45946,
0.46046,	0.46146,	0.46246,	0.46346,	0.46446,
0.46547,	0.46647,	0.46747,	0.46847,	0.46947,
0.47047,	0.47147,	0.47247,	0.47347,	0.47447,
0.47548,	0.47648,	0.47748,	0.47848,	0.47948,
	0.48048,	0.48148,	0.48248,	0.48348,
0.48448,	0.48549,	0.48649,	0.48749,	0.48849,
0.48949,	0.49049,	0.49149,	0.49249,	0.49349,
0.49449,	0.4955,	0.4965,	0.4975,	0.4985,
0.4995,	0.5005,	0.5015,	0.5025,	0.5035,
	0.5045,	0.50551,	0.50651,	0.50751,
0.50851,	0.50951,	0.51051,	0.51151,	0.51251,
0.51351,	0.51451,	0.51552,	0.51652,	0.51752,
0.51852,	0.51952,	0.52052,	0.52152,	0.52252,
0.52352,	0.52452,	0.52553,	0.52653,	0.52753,
	0.52853,	0.52953,	0.53053,	0.53153,

0.53253,	0.53353,	0.53453,	0.53554,	0.53654,
0.53754,	0.53854,	0.53954,	0.54054,	0.54154,
0.54254,	0.54354,	0.54454,	0.54555,	0.54655,
0.54755,	0.54855,	0.54955,	0.55055,	0.55155,
	0.55255,	0.55355,	0.55455,	0.55556,
0.55656,	0.55756,	0.55856,	0.55956,	0.56056,
0.56156,	0.56256,	0.56356,	0.56456,	0.56557,
0.56657,	0.56757,	0.56857,	0.56957,	0.57057,
0.57157,	0.57257,	0.57357,	0.57457,	0.57558,
	0.57658,	0.57758,	0.57858,	0.57958,
0.58058,	0.58158,	0.58258,	0.58358,	0.58458,
0.58559,	0.58659,	0.58759,	0.58859,	0.58959,
0.59059,	0.59159,	0.59259,	0.59359,	0.59459,
0.5956,	0.5966,	0.5976,	0.5986,	0.5996,
	0.6006,	0.6016,	0.6026,	0.6036,
0.6046,	0.60561,	0.60661,	0.60761,	0.60861,
0.60961,	0.61061,	0.61161,	0.61261,	0.61361,
0.61461,	0.61562,	0.61662,	0.61762,	0.61862,
0.61962,	0.62062,	0.62162,	0.62262,	0.62362,
	0.62462,	0.62563,	0.62663,	0.62763,
0.62863,	0.62963,	0.63063,	0.63163,	0.63263,
0.63363,	0.63463,	0.63564,	0.63664,	0.63764,
0.63864,	0.63964,	0.64064,	0.64164,	0.64264,
0.64364,	0.64464,	0.64565,	0.64665,	0.64765,
	0.64865,	0.64965,	0.65065,	0.65165,
0.65265,	0.65365,	0.65465,	0.65566,	0.65666,
0.65766,	0.65866,	0.65966,	0.66066,	0.66166,
0.66266,	0.66366,	0.66466,	0.66567,	0.66667,
0.66767,	0.66867,	0.66967,	0.67067,	0.67167,
	0.67267,	0.67367,	0.67467,	0.67568,
0.67668,	0.67768,	0.67868,	0.67968,	0.68068,
0.68168,	0.68268,	0.68368,	0.68468,	0.68569,
0.68669,	0.68769,	0.68869,	0.68969,	0.69069,
0.69169,	0.69269,	0.69369,	0.69469,	0.6957,
	0.6967,	0.6977,	0.6987,	0.6997,
0.7007,	0.7017,	0.7027,	0.7037,	0.7047,
0.70571,	0.70671,	0.70771,	0.70871,	0.70971,
0.71071,	0.71171,	0.71271,	0.71371,	0.71471,
0.71572,	0.71672,	0.71772,	0.71872,	0.71972,
	0.72072,	0.72172,	0.72272,	0.72372,
0.72472,	0.72573,	0.72673,	0.72773,	0.72873,
0.72973,	0.73073,	0.73173,	0.73273,	0.73373,
0.73473,	0.73574,	0.73674,	0.73774,	0.73874,
0.73974,	0.74074,	0.74174,	0.74274,	0.74374,
	0.74474,	0.74575,	0.74675,	0.74775,
0.74875,	0.74975,	0.75075,	0.75175,	0.75275,
0.75375,	0.75475,	0.75576,	0.75676,	0.75776,
0.75876,	0.75976,	0.76076,	0.76176,	0.76276,
0.76376,	0.76476,	0.76577,	0.76677,	0.76777,

	0.76877,	0.76977,	0.77077,	0.77177,	
0.77277,	0.77377,	0.77477,	0.77578,	0.77678,	
0.77778,	0.77878,	0.77978,	0.78078,	0.78178,	
0.78278,	0.78378,	0.78478,	0.78579,	0.78679,	
0.78779,	0.78879,	0.78979,	0.79079,	0.79179,	
	0.79279,	0.79379,	0.79479,	0.7958,	
0.7968,	0.7978,	0.7988,	0.7998,	0.8008,	
0.8018,	0.8028,	0.8038,	0.8048,	0.80581,	
0.80681,	0.80781,	0.80881,	0.80981,	0.81081,	
0.81181,	0.81281,	0.81381,	0.81481,	0.81582,	
	0.81682,	0.81782,	0.81882,	0.81982,	
0.82082,	0.82182,	0.82282,	0.82382,	0.82482,	
0.82583,	0.82683,	0.82783,	0.82883,	0.82983,	
0.83083,	0.83183,	0.83283,	0.83383,	0.83483,	
0.83584,	0.83684,	0.83784,	0.83884,	0.83984,	
	0.84084,	0.84184,	0.84284,	0.84384,	
0.84484,	0.84585,	0.84685,	0.84785,	0.84885,	
0.84985,	0.85085,	0.85185,	0.85285,	0.85385,	
0.85485,	0.85586,	0.85686,	0.85786,	0.85886,	
0.85986,	0.86086,	0.86186,	0.86286,	0.86386,	
	0.86486,	0.86587,	0.86687,	0.86787,	
0.86887,	0.86987,	0.87087,	0.87187,	0.87287,	
0.87387,	0.87487,	0.87588,	0.87688,	0.87788,	
0.87888,	0.87988,	0.88088,	0.88188,	0.88288,	
0.88388,	0.88488,	0.88589,	0.88689,	0.88789,	
	0.88889,	0.88989,	0.89089,	0.89189,	
0.89289,	0.89389,	0.89489,	0.8959,	0.8969,	
0.8979,	0.8989,	0.8999,	0.9009,	0.9019,	
0.9029,	0.9039,	0.9049,	0.90591,	0.90691,	
0.90791,	0.90891,	0.90991,	0.91091,	0.91191,	
	0.91291,	0.91391,	0.91491,	0.91592,	
0.91692,	0.91792,	0.91892,	0.91992,	0.92092,	
0.92192,	0.92292,	0.92392,	0.92492,	0.92593,	
0.92693,	0.92793,	0.92893,	0.92993,	0.93093,	
0.93193,	0.93293,	0.93393,	0.93493,	0.93594,	
	0.93694,	0.93794,	0.93894,	0.93994,	
0.94094,	0.94194,	0.94294,	0.94394,	0.94494,	
0.94595,	0.94695,	0.94795,	0.94895,	0.94995,	
0.95095,	0.95195,	0.95295,	0.95395,	0.95495,	
0.95596,	0.95696,	0.95796,	0.95896,	0.95996,	
	0.96096,	0.96196,	0.96296,	0.96396,	
0.96496,	0.96597,	0.96697,	0.96797,	0.96897,	
0.96997,	0.97097,	0.97197,	0.97297,	0.97397,	
0.97497,	0.97598,	0.97698,	0.97798,	0.97898,	
0.97998,	0.98098,	0.98198,	0.98298,	0.98398,	
	0.98498,	0.98599,	0.98699,	0.98799,	
0.98899,	0.98999,	0.99099,	0.99199,	0.99299,	
0.99399,	0.99499,	0.996,	0.997,	0.998,	
0.999,	1]],	array([[	0.94469,	0.94469,	0.94278,

0.94024,	0.93516,	0.93388,	0.93388,	0.93325,
0.93316,	0.93134,	0.93071,	0.93071,	0.93071,
0.93071,	0.92943,	0.9288,	0.92816,	0.92816,
0.92816,	0.92753,	0.92753,	0.92689,	0.92498,
	0.92371,	0.92371,	0.92308,	0.92308,
0.92244,	0.92244,	0.92244,	0.92244,	0.92117,
0.91926,	0.91863,	0.91799,	0.91672,	0.91672,
0.91672,	0.91545,	0.91545,	0.91545,	0.91481,
0.91418,	0.91418,	0.91291,	0.91291,	
	0.91291,	0.91227,	0.91227,	0.91163,
0.91163,	0.91036,	0.90973,	0.90973,	0.90973,
0.90909,	0.90909,	0.90846,	0.90718,	0.90718,
0.90655,	0.90591,	0.90591,	0.90591,	0.90591,
0.90528,	0.90528,	0.90401,	0.90273,	
	0.90273,	0.9021,	0.90083,	0.90083,
0.90019,	0.89943,	0.89892,	0.89892,	0.89828,
0.89701,	0.89701,	0.89638,	0.89383,	0.89383,
0.8932,	0.8932,	0.8932,	0.8932,	0.8932,
0.8932,	0.89278,	0.89193,	0.89193,	
	0.89193,	0.89193,	0.89065,	0.89065,
0.88938,	0.88908,	0.88811,	0.88748,	0.88684,
0.88684,	0.88647,	0.8862,	0.88557,	0.8843,
0.8843,	0.8843,	0.8843,	0.88366,	0.88366,
0.88366,	0.88366,	0.88366,	0.88357,	
	0.88303,	0.88303,	0.88239,	0.88239,
0.88175,	0.88175,	0.88175,	0.88175,	0.88172,
0.88112,	0.88112,	0.88112,	0.88048,	0.88048,
0.87985,	0.87921,	0.87921,	0.87858,	0.87858,
0.87794,	0.87794,	0.87794,	0.8773,	
	0.87667,	0.87603,	0.87467,	0.87349,
0.87349,	0.87349,	0.87349,	0.87285,	0.87285,
0.87222,	0.87158,	0.87158,	0.87095,	0.87095,
0.87095,	0.87095,	0.87095,	0.87095,	0.87095,
0.87095,	0.87095,	0.87095,	0.87095,	
	0.87095,	0.87095,	0.87095,	0.87095,
0.87095,	0.87095,	0.87095,	0.87095,	0.87095,
0.86953,	0.8684,	0.86816,	0.86777,	0.86713,
0.8665,	0.8665,	0.8665,	0.8665,	0.8665,
0.8665,	0.86586,	0.86523,	0.86523,	
	0.86459,	0.86332,	0.86332,	0.86268,
0.86261,	0.86205,	0.86205,	0.86205,	0.86205,
0.86205,	0.86141,	0.86141,	0.86141,	0.86141,
0.86141,	0.86141,	0.86078,	0.86078,	0.86078,
0.86078,	0.86078,	0.86078,	0.86064,	
	0.86,	0.8595,	0.8595,	0.8595,
0.8595,	0.8595,	0.8595,	0.8595,	0.8595,
0.8595,	0.8595,	0.8595,	0.8595,	0.8595,
0.85905,	0.85877,	0.85823,	0.85823,	0.85823,
0.8576,	0.8576,	0.8576,	0.8576,	

	0.85696,	0.85696,	0.85696,	0.85684,
0.85569,	0.85563,	0.85525,	0.85442,	0.85428,
0.85378,	0.85282,	0.85251,	0.85188,	0.85166,
0.8506,	0.85053,	0.84997,	0.84997,	0.84956,
0.84933,	0.84933,	0.84933,	0.84933,	
	0.8487,	0.8487,	0.8487,	0.8487,
0.8487,	0.8487,	0.84806,	0.84796,	0.84744,
0.84743,	0.84743,	0.84679,	0.84615,	0.84615,
0.84615,	0.84615,	0.84615,	0.84615,	0.84615,
0.84615,	0.84508,	0.84488,	0.84488,	
	0.84488,	0.84465,	0.84434,	0.84365,
0.84361,	0.84361,	0.84234,	0.84213,	0.8417,
0.8417,	0.84144,	0.84107,	0.84105,	0.84062,
0.83917,	0.83806,	0.83732,	0.83709,	0.83598,
0.83598,	0.83598,	0.83598,	0.83471,	
	0.83471,	0.83471,	0.83471,	0.83471,
0.8339,	0.83344,	0.83329,	0.83253,	0.83217,
0.83138,	0.83114,	0.8309,	0.83026,	0.82971,
0.82899,	0.82899,	0.82899,	0.82899,	0.82899,
0.82899,	0.82772,	0.82772,	0.82772,	
	0.82708,	0.82708,	0.82659,	0.82645,
0.82645,	0.82547,	0.82517,	0.82517,	0.8249,
0.82454,	0.82454,	0.82454,	0.82454,	0.82419,
0.82327,	0.82327,	0.82263,	0.82263,	0.82263,
0.82222,	0.822,	0.822,	0.822,	
	0.822,	0.82143,	0.82136,	0.82136,
0.82136,	0.82136,	0.82136,	0.82014,	0.82009,
0.82009,	0.82009,	0.81982,	0.81946,	0.81843,
0.81818,	0.81818,	0.81774,	0.81755,	0.81522,
0.81459,	0.8131,	0.8131,	0.81246,	
	0.81199,	0.81182,	0.81182,	0.81182,
0.81182,	0.81119,	0.81055,	0.80992,	0.80887,
0.80794,	0.80737,	0.80737,	0.80674,	0.80568,
0.80547,	0.80462,	0.80348,	0.80292,	0.80292,
0.80292,	0.80279,	0.80222,	0.80186,	
	0.80165,	0.80165,	0.80093,	0.80025,
0.79953,	0.79923,	0.79879,	0.79784,	0.79768,
0.7972,	0.79657,	0.79657,	0.79604,	0.79485,
0.79466,	0.79402,	0.79402,	0.79402,	0.79402,
0.79402,	0.79402,	0.79402,	0.79339,	
	0.79339,	0.79241,	0.79196,	0.79085,
0.79025,	0.79021,	0.78957,	0.78957,	0.78957,
0.78885,	0.78854,	0.7883,	0.78686,	0.78605,
0.78512,	0.78449,	0.78449,	0.78395,	0.78305,
0.78205,	0.78156,	0.78004,	0.77986,	
	0.77963,	0.7794,	0.77756,	0.77686,
0.77622,	0.77622,	0.77622,	0.77622,	0.77622,
0.77622,	0.77622,	0.77476,	0.77432,	0.77408,
0.77308,	0.77305,	0.77305,	0.77159,	0.77108,
0.77079,	0.7705,	0.76987,	0.76923,	

	0.76796,	0.76766,	0.76531,	0.76478,
0.76451,	0.76414,	0.76399,	0.76302,	0.76213,
0.7616,	0.76158,	0.76127,	0.76097,	0.76097,
0.76033,	0.75766,	0.75635,	0.75588,	0.75397,
0.75397,	0.75358,	0.75249,	0.75184,	
	0.75143,	0.75076,	0.74913,	0.74709,
0.74634,	0.74526,	0.74444,	0.7438,	0.7438,
0.7438,	0.74189,	0.74156,	0.74072,	0.74047,
0.7402,	0.73988,	0.73939,	0.73872,	0.73872,
0.73816,	0.73744,	0.73733,	0.73681,	
	0.73628,	0.73481,	0.73363,	0.73236,
0.73127,	0.7306,	0.73041,	0.73012,	0.72983,
0.72951,	0.72918,	0.72889,	0.72791,	0.72791,
0.72791,	0.72764,	0.72709,	0.72664,	0.72664,
0.72634,	0.7257,	0.72409,	0.724,	
	0.72378,	0.72356,	0.72225,	0.72219,
0.72219,	0.72114,	0.71964,	0.71901,	0.71901,
0.71862,	0.71754,	0.71584,	0.71542,	0.71385,
0.71329,	0.71197,	0.71159,	0.71097,	0.71,
0.70947,	0.70947,	0.70722,	0.70465,	
	0.7022,	0.70057,	0.69988,	0.6975,
0.69612,	0.69512,	0.69361,	0.69301,	0.69276,
0.69104,	0.68785,	0.68566,	0.68382,	0.683,
0.68193,	0.68153,	0.68027,	0.67982,	0.6787,
0.67803,	0.67682,	0.67578,	0.67473,	
	0.6726,	0.67193,	0.67002,	0.66845,
0.66792,	0.66752,	0.66688,	0.66477,	0.66374,
0.66332,	0.66224,	0.66052,	0.6588,	0.65804,
0.65623,	0.65287,	0.65099,	0.65099,	0.64844,
0.64816,	0.6471,	0.64454,	0.64336,	
	0.64062,	0.63981,	0.63658,	0.63446,
0.63251,	0.63112,	0.62873,	0.62762,	0.62572,
0.62045,	0.62005,	0.6193,	0.61602,	0.61475,
0.61368,	0.61135,	0.60909,	0.60623,	0.60388,
0.60314,	0.6016,	0.60035,	0.5978,	
	0.59392,	0.59135,	0.58774,	0.58584,
0.58377,	0.58133,	0.58048,	0.5786,	0.57525,
0.57416,	0.57152,	0.56863,	0.56646,	0.56516,
0.56414,	0.56234,	0.55888,	0.55702,	0.55321,
0.55092,	0.54595,	0.54367,	0.54047,	
	0.53627,	0.53473,	0.53192,	0.52841,
0.52662,	0.52505,	0.52366,	0.52162,	0.52086,
0.52003,	0.51742,	0.51373,	0.51147,	0.50889,
0.50697,	0.50481,	0.50194,	0.49698,	0.4943,
0.49121,	0.48977,	0.48658,	0.48232,	
	0.47895,	0.47759,	0.47525,	0.47171,
0.46895,	0.4655,	0.46302,	0.46157,	0.45978,
0.45682,	0.45379,	0.45018,	0.44637,	0.44374,
0.44216,	0.43965,	0.43777,	0.43491,	0.42657,

0.42421,	0.42073,	0.41606,	0.40941,		
	0.40503,	0.40057,	0.39817,	0.39456,	
0.38963,	0.38387,	0.379,	0.37545,	0.37383,	
0.37033,	0.36552,	0.36162,	0.35683,	0.3534,	
0.34511,	0.33996,	0.33159,	0.32223,	0.31575,	
0.31029,	0.30562,	0.29712,	0.2918,		
	0.28683,	0.28092,	0.276,	0.27133,	
0.26503,	0.25814,	0.2485,	0.24249,	0.23781,	
0.22874,	0.22329,	0.21805,	0.21268,	0.20525,	
0.20155,	0.19878,	0.19396,	0.18678,	0.18209,	
0.17771,	0.17175,	0.16353,	0.16074,		
	0.15683,	0.15148,	0.14591,	0.14338,	
0.13793,	0.134,	0.132,	0.12924,	0.1212,	
0.11644,	0.11427,	0.11394,	0.1116,	0.10885,	
0.1055,	0.10168,	0.098229,	0.096374,	0.093733,	
0.091704,	0.08689,	0.083654,	0.08099,		
	0.076697,	0.076287,	0.07412,	0.07317,	
0.070204,	0.067998,	0.067555,	0.066071,	0.065628,	
0.063888,	0.062472,	0.060002,	0.057695,	0.057049,	
0.055308,	0.053316,	0.051778,	0.049422,	0.047673,	
0.045704,	0.044318,	0.043622,	0.043294,		
	0.040739,	0.038875,	0.037621,	0.036127,	
0.034691,	0.034266,	0.03409,	0.033914,	0.033738,	
0.032394,	0.032034,	0.031066,	0.030539,	0.029862,	
0.029624,	0.029386,	0.028858,	0.027552,	0.026949,	
0.026474,	0.025945,	0.024003,	0.023318,		
	0.021438,	0.020209,	0.019883,	0.019058,	
0.018949,	0.01884,	0.018731,	0.018622,	0.018514,	
0.017743,	0.017338,	0.016644,	0.015607,	0.015188,	
0.014533,	0.013932,	0.013746,	0.013561,	0.013375,	
0.012419,	0.012012,	0.011341,	0.0111,		
	0.010858,	0.010486,	0.0095345,	0.0092253,	
0.008916,	0.0073811,	0.0069307,	0.0065913,	0.0059882,	
0.0049968,	0.004818,	0.0046391,	0.0044602,	0.0034909,	
0.0026103,	0.0019263,	0.0016151,	0.0013142,	0.0011563,	
0.0010221,	0.00088793,	0.00075373,	0,		
	0,	0,	0,	0,	
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,					
	0,	0,	0,	0,	
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,
0,					
	0,	0,	0,	0,	
0,	0,	0,	0,	0,	0,
0,	0,	0,	0,	0,	0,

```
0, 0, 0, 0, 0, 0,
0,
0, 0, 0, 0,
0, 0, 0, 0, 0,
0, 0, 0, 0, 0,
0, 0, 0, 0,
0,
0, 0, 0, 0,
0, 0, 0, 0,
0, 0, 0, 0,
0,
0, 0, 0, 0,
0, 0, 0, 0,
0, 0, 0, 0,
0,
0, 0, 0, 0,
0, 0, 0, 0,
0, 0, 0, 0,
0,
0, 0, 0, 0,
0, 0, 0, 0,
0, 0, 0, 0,
0,
0, 0, 0, 0, 0, 0,
0]]), 'Confidence', 'Recall']]
fitness: 0.43226485248149293
keys: ['metrics/precision(B)', 'metrics/recall(B)',
'metrics/mAP50(B)', 'metrics/mAP50-95(B)']
maps: array([ 0.43226])
names: {0: 'license_plate'}
nt_per_class: array([1573])
nt_per_image: array([1073])
results_dict: {'metrics/precision(B)': 0.8507367645102252,
'metrics/recall(B)': 0.8029243483788938, 'metrics/mAP50(B)':
0.8551930213850748, 'metrics/mAP50-95(B)': 0.43226485248149293,
'fitness': 0.43226485248149293}
save_dir: PosixPath('/kaggle/working/runs/detect/val')
speed: {'preprocess': 0.7283094753086898, 'inference':
17.187312374657907, 'loss': 0.0005276262903674345, 'postprocess':
0.714633784714915}
stats: {'tp': [], 'conf': [], 'pred_cls': [], 'target_cls': [],
'target_img': []}
```

Load Training Results

```
def load_training_results():
    csv_path = "/kaggle/working/runs/detect/train/results.csv"
    df = pd.read_csv(csv_path)
    return df

results_df = load_training_results()
```

Plot Training Metrics

results_df

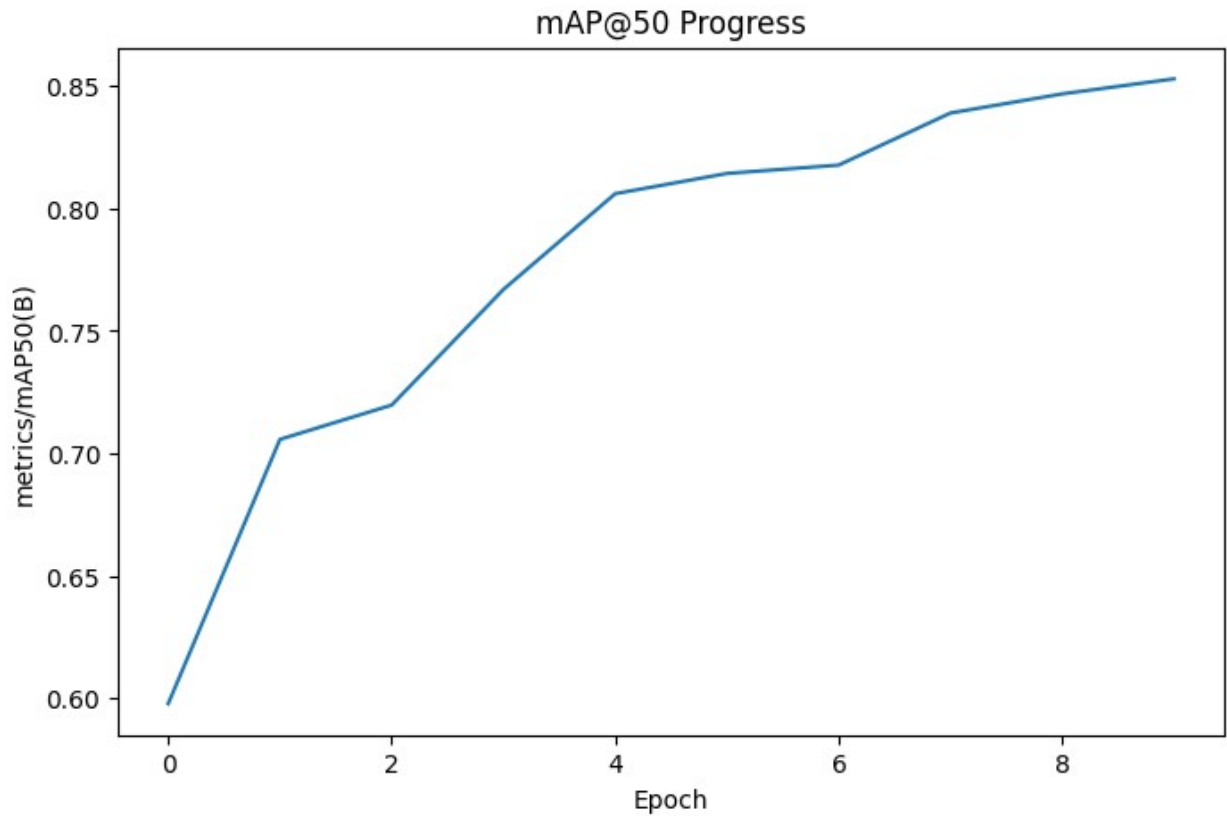
	epoch	time	train/box_loss	train/cls_loss	train/df_l_loss	\
0	1	635.412	1.29604	0.81038	1.30192	
1	2	1263.150	1.23945	0.65711	1.26561	
2	3	1886.230	1.20588	0.60550	1.24525	
3	4	2508.780	1.17869	0.56661	1.22724	
4	5	3129.150	1.15516	0.52911	1.20746	
5	6	3749.550	1.13595	0.50324	1.19665	
6	7	4369.610	1.11969	0.48408	1.18002	
7	8	4990.540	1.10209	0.46306	1.17119	
8	9	5613.910	1.08711	0.44461	1.16025	
9	10	6237.130	1.06537	0.42415	1.15006	

	metrics/precision(B)	metrics/recall(B)	metrics/mAP50(B)	\
0	0.62853	0.63064	0.59779	
1	0.75068	0.68849	0.70571	
2	0.75830	0.71901	0.71970	
3	0.80864	0.73609	0.76692	
4	0.82521	0.76234	0.80602	
5	0.81110	0.78067	0.81429	
6	0.81399	0.78172	0.81771	
7	0.83328	0.80992	0.83897	
8	0.84306	0.80252	0.84678	
9	0.84828	0.79975	0.85292	

	metrics/mAP50-95(B)	val/box_loss	val/cls_loss	val/df_l_loss
lr/pg0 \				
0	0.26939	1.81232	1.15956	1.40818
0.000666				
1	0.31295	1.72136	1.08387	1.33741
0.001201				
2	0.33927	1.65694	0.94309	1.32795
0.001604				
3	0.35774	1.60640	0.89606	1.26089
0.001406				
4	0.38115	1.59746	0.83395	1.27028
0.001208				
5	0.40520	1.53560	0.79000	1.23807
0.001010				
6	0.40823	1.51899	0.77062	1.21317
0.000812				
7	0.41508	1.52144	0.75130	1.22395
0.000614				
8	0.42680	1.48970	0.71152	1.21536
0.000416				
9	0.43253	1.47999	0.71141	1.19550
0.000218				

	lr/pg1	lr/pg2
0	0.000666	0.000666
1	0.001201	0.001201
2	0.001604	0.001604
3	0.001406	0.001406
4	0.001208	0.001208
5	0.001010	0.001010
6	0.000812	0.000812
7	0.000614	0.000614
8	0.000416	0.000416
9	0.000218	0.000218

```
def plot_training_metric(df, column_name, title):  
    plt.figure(figsize=(8,5))  
    plt.plot(df[column_name])  
    plt.title(title)  
    plt.xlabel("Epoch")  
    plt.ylabel(column_name)  
    plt.show()  
  
plot_training_metric(  
    results_df,  
    "metrics/mAP50(B)",  
    "mAP@50 Progress"  
)
```



```
import matplotlib.pyplot as plt

# -----
# Generic Plot Function
# -----
def plot_training_metric_new(df, columns, title, ylabel="Value"):
    plt.figure(figsize=(10, 6))

    # If single column passed
    if isinstance(columns, str):
        columns = [columns]

    # Plot all columns
    for column in columns:
        plt.plot(
            df[column],
            marker='o',
            linewidth=2,
            label=column
        )

    plt.title(title, fontsize=16)
```

```

plt.xlabel("Epoch", fontsize=12)
plt.ylabel(ylabel, fontsize=12)
plt.grid(True)
plt.legend()
plt.show()

# -----
# 1. Training Loss Graph
# -----
plot_training_metric_new(
    results_df,
    [
        "train/box_loss",
        "train/cls_loss",
        "train/dfl_loss"
    ],
    "Training Loss Progress",
    "Loss"
)

# -----
# 2. Validation Loss Graph
# -----
plot_training_metric_new(
    results_df,
    [
        "val/box_loss",
        "val/cls_loss",
        "val/dfl_loss"
    ],
    "Validation Loss Progress",
    "Loss"
)

# -----
# 3. Precision & Recall Graph
# -----
plot_training_metric_new(
    results_df,
    [
        "metrics/precision(B)",
        "metrics/recall(B)"
    ],

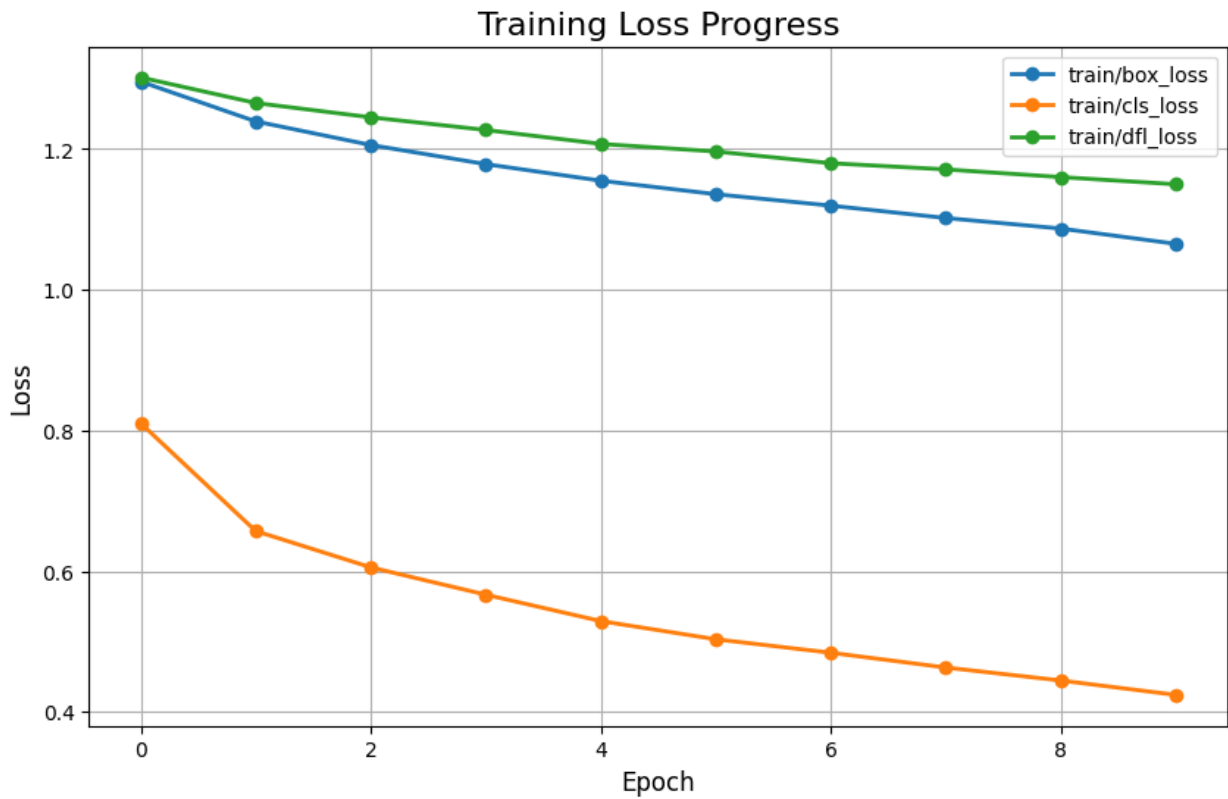
```

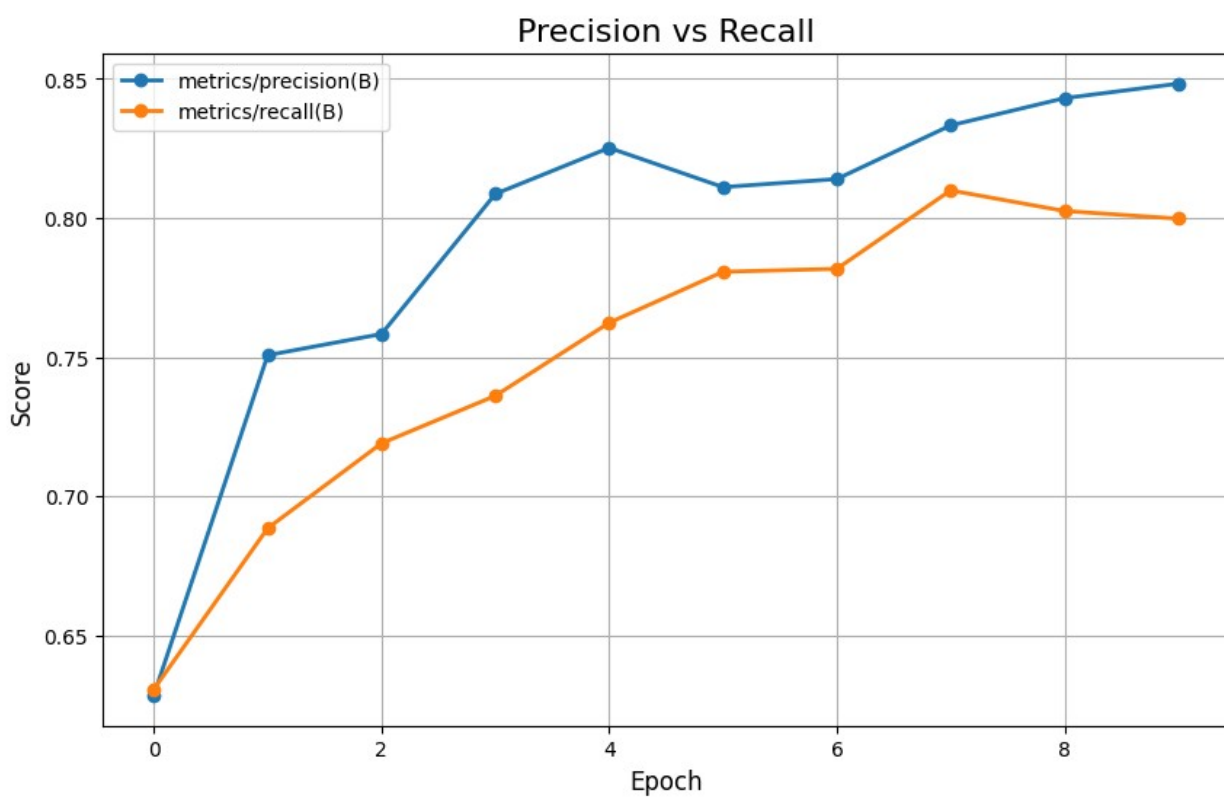
```

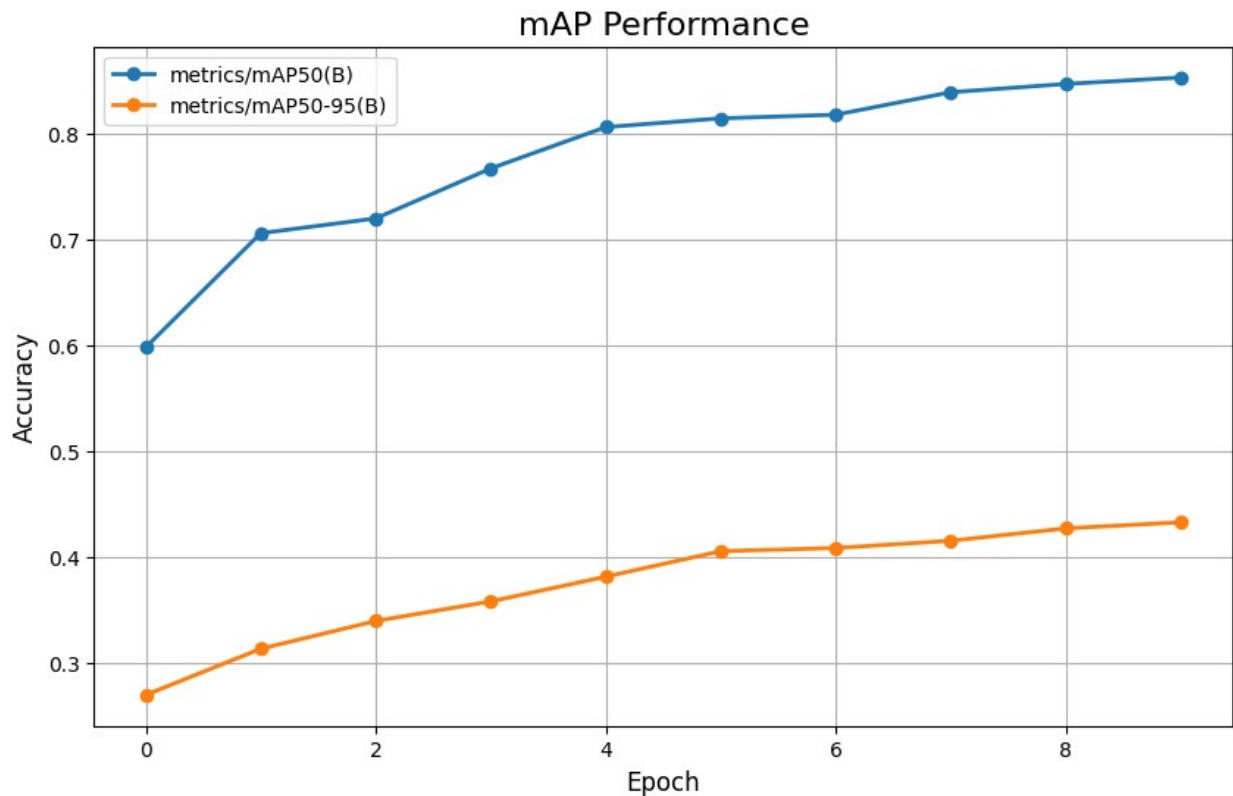
    "Precision vs Recall",
    "Score"
)

# -----
# 4. mAP Performance Graph
# -----
plot_training_metric_new(
    results_df,
    [
        "metrics/mAP50(B)",
        "metrics/mAP50-95(B)"
    ],
    "mAP Performance",
    "Accuracy"
)

```







SECTION 12 — Inference Pipeline

Predict Image

```
def predict_image(model, image_path):  
    results = model(image_path)  
    return results
```

SECTION 13 — License Plate Blurring

Blur Function

```
def blur_license_plate(image, box):  
    x1, y1, x2, y2 = map(int, box)  
    roi = image[y1:y2, x1:x2]  
    if roi.size == 0:  
        return image  
    blurred_roi = cv2.GaussianBlur(  

```

```

        roi,
        (51,51),
        30
    )

    image[y1:y2, x1:x2] = blurred_roi

    return image

```

Full Image Processing Pipeline

```

def extract_text(image):
    import pytesseract

    gray = cv2.cvtColor(
        image,
        cv2.COLOR_BGR2GRAY
    )

    # ENLARGE IMAGE
    gray = cv2.resize(
        gray,
        None,
        fx=3,
        fy=3,
        interpolation=cv2.INTER_CUBIC
    )

    # THRESHOLDING
    gray = cv2.threshold(
        gray,
        0,
        255,
        cv2.THRESH_BINARY + cv2.THRESH_OTSU
    )[1]

    text = pytesseract.image_to_string(
        gray,
        config='--psm 7'
    )

    return text.strip()

def process_image(model, image_path):
    import pytesseract

    image = cv2.imread(image_path)

```

```

results = model(image, conf=0.5)[0]

#results = model(image)[0]

#for box in results.bboxes.xyxy:
for box in results.bboxes.xyxy.cpu().numpy():

    image = blur_license_plate(
        image,
        box
    )

    return image

def validate_blur_ocr(model, image_path):

    import pytesseract

    image = cv2.imread(image_path)

    results = model(image, conf=0.5)[0]

    for box in results.bboxes.xyxy.cpu().numpy():

        x1, y1, x2, y2 = map(int, box)

        # ORIGINAL ROI
        original_roi = image[y1:y2, x1:x2]

        if original_roi.size == 0:
            continue

        # OCR BEFORE BLUR
        original_text = extract_text(original_roi)

        # APPLY BLUR
        blurred_roi = cv2.GaussianBlur(
            original_roi,
            (51,51),
            30
        )

        # OCR AFTER BLUR
        blurred_text = extract_text(blurred_roi)

        # VISUALIZATION
        plt.figure(figsize=(12,5))

        plt.subplot(1,2,1)

        plt.imshow(

```

```

        cv2.cvtColor(original_roi, cv2.COLOR_BGR2RGB)
    )

    plt.title(
        f"Before Blur OCR:\n{original_text}"
    )

    plt.axis("off")

    plt.subplot(1,2,2)

    plt.imshow(
        cv2.cvtColor(blurred_roi, cv2.COLOR_BGR2RGB)
    )

    plt.title(
        f"After Blur OCR:\n{blurred_text}"
    )

    plt.axis("off")

    plt.show()

    print("="*50)

    print("OCR Before Blur:", original_text)

    print("OCR After Blur:", blurred_text)

    print("="*50)

```

SECTION 14 — Before vs After Comparison

Visualization Function

```

def compare_before_after_with(model, image_path):
    original = cv2.imread(image_path)

    original = cv2.cvtColor(
        original,
        cv2.COLOR_BGR2RGB
    )

    processed = process_image(
        model,
        image_path
    )

    cv2.imwrite(

```

```
        "/kaggle/working/blurred_output.jpg",
        processed
    )

    processed = cv2.cvtColor(
        processed,
        cv2.COLOR_BGR2RGB
    )

    plt.figure(figsize=(14,7))
    plt.subplot(1,2,1)
    plt.imshow(original)
    plt.title("Original Image")
    plt.axis("off")
    plt.subplot(1,2,2)
    plt.imshow(processed)
    plt.title("Blurred Image")
    plt.axis("off")
    plt.show()
```

Run Demo

```
def run_demo_with_ocr(paths, model):
    image_name = random.choice(
        os.listdir(paths["test_images"])
    )

    image_path = os.path.join(
        paths["test_images"],
        image_name
    )

    validate_blur_ocr(
        model,
        image_path
    )

run_demo_with_ocr(paths, model)
```

0: 480x640 1 license_plate, 28.6ms
Speed: 2.9ms preprocess, 28.6ms inference, 1.4ms postprocess per image
at shape (1, 3, 480, 640)

Before Blur OCR:
NO. NELET



After Blur OCR:
Se



```
=====
OCR Before Blur: NO. NELET
OCR After Blur: Se
=====
```

```
def run_demo(paths, model):
    image_name = random.choice(
        os.listdir(paths["test_images"])
    )

    image_path = os.path.join(
        paths["test_images"],
        image_name
    )

    compare_before_after_with(
        model,
        image_path
    )

run_demo(paths, model)
```

0: 480x640 1 license_plate, 27.7ms
Speed: 2.8ms preprocess, 27.7ms inference, 1.4ms postprocess per image
at shape (1, 3, 480, 640)

Original Image



Blurred Image



```
model.save(  
    "/kaggle/working/license_plate_detector.pt"  
)
```

SECTION 15 — Error Analysis

```
def error_analysis():  
    print("""  
    Common Failure Cases:  
  
    1. Low-light conditions  
    2. Motion blur  
    3. Distant vehicles  
    4. Partial occlusion  
    5. Extreme camera angles  
  
    Future Improvements:  
  
    - Add nighttime augmentation  
    - Increase image resolution  
    - Add difficult samples  
    - Fine-tune larger YOLO models  
    """)  
  
error_analysis()
```

Common Failure Cases:

1. Low-light conditions
2. Motion blur
3. Distant vehicles
4. Partial occlusion
5. Extreme camera angles

Future Improvements:

- Add nighttime augmentation
- Increase image resolution
- Add difficult samples
- Fine-tune larger YOLO models

OCR Verification After Blurring

OCR was tested before and after plate blurring.

Results showed:

- Original image → OCR successfully extracted text
- Blurred image → OCR failed to read plate number

This confirms privacy preservation effectiveness.

SECTION 16 — Final Performance summary

Final Performance Summary

The YOLOv8m model achieved strong detection accuracy while maintaining near real-time inference performance.

The system successfully:

- detected vehicle license plates,
- applied localized Gaussian blur,
- preserved surrounding image usability,
- supported privacy-preserving workflows.

The solution demonstrates strong potential for:

- traffic surveillance,
- smart city systems,
- insurance analytics,
- privacy-compliant data processing.

SECTION 17 — Recommendations

```
def recommendations():  
    print("""  
    Strategic Recommendations:
```

1. Deploy using TensorRT
 2. Use ONNX for edge devices
 3. Add continuous retraining
 4. Integrate privacy filtering before cloud upload
 5. Use adaptive Gaussian blur kernels
- ```
"""
```

```
recommendations()
```

#### Strategic Recommendations:

1. Deploy using TensorRT
2. Use ONNX for edge devices
3. Add continuous retraining
4. Integrate privacy filtering before cloud upload
5. Use adaptive Gaussian blur kernels

## SECTION 18 — Business Questions

### Q1. How do you convert normalized YOLO bounding box coordinates into pixel coordinates suitable for drawing rectangles with OpenCV?

YOLO stores all bounding box coordinates as values normalized between 0 and 1, relative to image width and height. This design makes annotations resolution-independent — the same label file works for a 640×480 image or a 4K frame. However, OpenCV's drawing and cropping functions require absolute pixel positions. The conversion involves four straightforward transformations. Given a normalized bounding box (center\_x, center\_y, width, height) and the actual image dimensions (img\_width, img\_height):

```
x1 = int((center_x - width / 2) * img_width) # left edge
```

```
y1 = int((center_y - height / 2) * img_height) # top edge
```

```
x2 = int((center_x + width / 2) * img_width) # right edge
```

```
y2 = int((center_y + height / 2) * img_height) # bottom edge
```

This is implemented in the project as `yolo_to_pixel()`, which is called both during pre-training bounding box visualization (Section 9) and during the post-inference blurring pipeline (Section 13). The resulting (x1, y1, x2, y2) format represents the top-left and bottom-right corners of the rectangle — exactly what `cv2.rectangle()` and NumPy array slicing (`image[y1:y2, x1:x2]`) expect.

### Q2. Write a function to overlay bounding boxes onto raw images. Why is this step critical before starting the training process?

The `draw_bounding_boxes()` function implemented in Section 9 reads each YOLO label file, calls `yolo_to_pixel()` for each annotation, and draws a green rectangle (`cv2.rectangle` with color (0, 255, 0)) over the corresponding region of the image before displaying it with `matplotlib`.

Why this step is non-negotiable before training:

- **Annotation Sanity Check:** YOLO label files are plain text — a single transcription error (e.g., swapped width and height, or a value >1) produces a visually obvious misaligned box. Catching this before training saves hours of GPU time.
- **Confirms Coordinate System:** Different annotation tools (LabelImg, Roboflow, CVAT) sometimes use slightly different coordinate conventions. Visual inspection instantly confirms that center-based normalized coordinates are being interpreted correctly.
- **Reveals Data Quality Issues:** Visualisation exposes duplicate boxes, boxes that clip at image edges, or plates that were missed entirely — all of which degrade model performance if uncorrected.
- **Validates Image-Label Pairing:** The `verify_image_label_pairs()` function checks that every image has a matching label file, but only visual inspection confirms the boxes actually land on the plates, not on random vehicle regions.

In short, this step converts a silent data integrity assumption into a verified fact. A model trained on misaligned annotations will learn incorrect spatial priors that are nearly impossible to diagnose from loss curves alone.

### **Q3. How did annotation quality impact the final blurring accuracy?**

Annotation quality acts as the upstream bottleneck in this pipeline — the model can only learn to locate what the annotations correctly describe. In this project, annotation quality impacted outcomes across three dimensions:

#### **1. Detection Confidence & Bounding Box Tightness:**

Annotations in the dataset were created at varying levels of tightness — some boxes closely wrapped the plate, while others included a margin of surrounding bodywork. This directly influenced whether the Gaussian blur fully covered the plate text or slightly undershot it. A tightly annotated model produces precise ROI extraction, which is critical because an under-blurred plate can still be read by OCR engines.

#### **1. Class Consistency:**

Since this is a single-class problem (class 0: `license_plate`), mislabelled or missing annotations introduce false negatives during training — the model 'sees' a plate but receives no gradient signal to detect it, effectively teaching it to ignore that configuration. The `verify_image_label_pairs()` function was used to confirm that all training images had corresponding label files, reducing this risk.

#### **1. Edge Cases & Generalization:**

Images where plates were partially occluded, at extreme angles, or in low contrast were the most annotation-sensitive. Inconsistently labelled challenging examples (labelled in some images but skipped in others) caused the model to develop inconsistent confidence in those scenarios — reflected in lower detection rates at the `conf=0.5` threshold used in `process_image()`.

#### **Q4. How did you preprocess images before feeding them into YOLOv8 without losing fine-grained plate details?**

YOLOv8 handles the majority of preprocessing internally, but the choices made at the configuration level directly determine whether fine-grained plate details are preserved:

Input Resolution — `imgsz=640`:

Images were resized to 640×640 pixels as required by the YOLOv8 input pipeline. This resolution was deliberately chosen because it provides sufficient spatial density to resolve small license plate characters, even when the plate occupies only a small fraction of the full frame. Higher resolutions (e.g., 1280) could marginally improve small-plate detection but significantly increase memory usage and inference time.

Augmentation Disabled — `augment=False`, `mosaic=0`, `mixup=0`:

Standard augmentations like mosaic (which tiles four images into one) and mixup (which blends two images) were explicitly disabled. For small, text-bearing objects like license plates, mosaic creates unnatural scale distortions and mixup blends pixel values, both of which can obscure the fine character-level detail that the model needs to localise correctly.

Backbone Freezing — `freeze=5`:

The first 5 layers of the YOLOv8 backbone (pre-trained on COCO) were frozen during training. These early layers already contain strong low-level feature detectors (edges, corners, fine textures) learned from millions of images. Freezing them preserves this fine-grained detection capability while allowing the deeper layers to specialise on license plate shapes and positions.

AMP Training — `amp=True`:

Automatic Mixed Precision (AMP) uses 16-bit floating point for most operations while maintaining 32-bit precision for critical computations. This allows larger effective batch sizes and faster training without any loss in detection precision.

#### **Q5. Why did you choose YOLOv8 for license plate detection instead of a two-stage detector?**

Two-stage detectors (Faster R-CNN, Mask R-CNN) follow a pipeline of first generating region proposals and then classifying each proposal. While highly accurate, this architecture has fundamental limitations that disqualify it for this use case:

Criterion Two-Stage (Faster R-CNN) YOLOv8 (One-Stage)

Inference Speed ~5–10 FPS (CPU) ~30–100+ FPS (GPU)

Pipeline RPN → Classify Single forward pass

Real-Time Deployment Not feasible Fully supported

Edge Device Export Complex ONNX / TensorRT ready

Training Complexity High Moderate

mAP Accuracy High Comparable on single-class

YOLOv8 processes the entire image in a single neural network pass, making it inherently faster and well-suited for real-time traffic camera feeds where latency is a hard constraint. For a single-class detection task (license plates), the accuracy penalty of switching from a two-stage to a one-stage detector is negligible, while the speed and deployment advantages are substantial.

**Q6. Which YOLOv8 variant did you select, and how did it balance speed and accuracy for this use case?**

The project selected YOLOv8m (medium variant), loaded via `YOLO("yolov8m.pt")`. This choice was made after considering the full variant spectrum:

Variant Parameters [mAP@50-95](#) Speed (ms) Use Case

YOLOv8n 3.2M 37.3 ~1.8ms IoT edge

YOLOv8s 11.2M 44.9 ~2.4ms Mobile apps

YOLOv8m ✓ 25.9M 50.2 ~5.0ms Balanced (selected)

YOLOv8l 43.7M 52.9 ~7.7ms High-accuracy

YOLOv8x 68.2M 53.9 ~13.7ms Research

YOLOv8m at 25.9M parameters provides the best balance for this workload. It is substantially more capable than nano/small variants at detecting small plate regions at various distances and angles, while remaining fast enough for near-real-time processing and exportable to TensorRT/ONNX for production deployment. The nano variant would sacrifice too much accuracy on distant or partially occluded plates; the large/xlarge variants add marginal mAP improvements at a disproportionate inference cost.

**Q7. How did license plate characteristics influence your choice of model scale?**

License plates have a distinctive set of visual properties that drove the model selection decision:

Small Object Size:

In a typical traffic surveillance frame, a license plate may occupy only 1–5% of the total image area. Smaller YOLO variants (nano, small) use fewer feature pyramid levels and shallower feature maps, which means they struggle to detect objects this small reliably. The medium variant's deeper architecture preserves small-object feature resolution better through its P3/P4/P5 detection heads.

High Aspect Ratio:

License plates are strongly rectangular — typically a 4:1 or 5:1 width-to-height ratio. Analysis of the training bounding box distributions (`plot_bbox_distribution()` in Section 7) confirmed that normalized widths cluster around 0.15–0.25 while heights cluster around 0.04–0.08. This elongated shape requires anchor configurations that can match wide, shallow bounding boxes — something the medium variant handles more robustly due to its richer feature representation.

Text-Level Detail:

Accurate localisation must be tight enough so that the applied Gaussian blur fully covers all alphanumeric characters. A model that outputs a bounding box slightly smaller than the actual plate will leave visible characters at the edges — a direct privacy failure. The medium variant's additional capacity helps it learn tighter, more precise localisations compared to lighter variants.

#### Backbone Freeze Compatibility:

The decision to freeze the first 5 backbone layers (freeze=5) was only viable because YOLOv8m's pre-trained weights already contain rich low-level feature detectors from COCO training. A nano model with fewer parameters has less generalizable frozen features, making the freeze strategy less effective.

#### **Q8. How did you apply blurring only within the detected bounding box?**

The blurring pipeline is implemented in `blur_license_plate()` (Section 13) and consists of three precise steps:

##### Step 1 — Extract the Region of Interest (ROI):

The model's output boxes (`results.bboxes.xyxy`) are already in pixel coordinates in `xyxy` format (`x1, y1, x2, y2`). These coordinates are converted to integers and used to slice the numpy array representing the image:

```
roi = image[y1:y2, x1:x2]
```

This creates a direct view into the image memory — only the plate region, with all surrounding pixels untouched.

##### Step 2 — Apply Gaussian Blur to the ROI Only:

A Gaussian Blur with kernel size (51, 51) and sigma=30 is applied exclusively to the extracted ROI:

```
blurred_roi = cv2.GaussianBlur(roi, (51, 51), 30)
```

The 51×51 kernel is deliberately large — it averages each pixel over a 51-pixel neighborhood, creating a heavily smoothed result that destroys character legibility while preserving the general shape and color of the plate region. A smaller kernel (e.g., 15×15) would produce insufficient blur that modern OCR engines could still partially decode.

##### Step 3 — Write the Blurred ROI Back:

The blurred ROI is written back into the exact same pixel region of the original image array:

```
image[y1:y2, x1:x2] = blurred_roi
```

Because this is a direct NumPy array assignment, only pixels within the bounding box coordinates are modified. All pixels outside that region — the vehicle body, road, background — remain at their original values. The ROI size guard (if `roi.size == 0`: return image) prevents crashes when the model produces degenerate boxes at image edges.

The full `process_image()` function runs this pipeline sequentially for every detected box in a frame, handling the case of multiple plates in a single image (e.g., convoy scenes) via a for loop over `results.bboxes.xyxy`.

## SECTION 19 — Conclusion

```
def project_conclusion():
 print("""
 Project Conclusion:

 Successfully developed an automated
 license plate detection and blurring
 pipeline using YOLOv8.

 Key Achievements:

 - Accurate license plate detection
 - Real-time inference capability
 - Privacy-preserving anonymization
 - Scalable deployment architecture
 """)

project_conclusion()
```